

Traceable Registered Functional Encryption for Unbounded Inner Product in Web Service Platforms

Jinmei Tian , Jianting Ning , Senior Member, IEEE, Jian Shen , Member, IEEE, Kai Zhang , Member, IEEE, Weijing You , and Lefeng Zhang 

Abstract—Web service platforms must not only ensure the computability of the encrypted data but also support the traceability of data leakage when sharing high-dimensional sensitive data. Functional encryption (FE) enables designated functional operations on ciphertexts without revealing the original plaintext information, making it a key technology for privacy-preserving computation in Web scenarios. However, most existing FE schemes are limited by fixed vector dimensions or reliance on indistinguishability obfuscation ($i\mathcal{O}$), which hinders the simultaneous achievement of unbounded inner product evaluation and effective tracing of malicious users. To overcome these limitations, we present a novel framework called traceable registered functional encryption for unbounded inner product (TRFE-UIP). Our framework eliminates the need for preset dimensional bounds and supports unbounded inner product operations without $i\mathcal{O}$. Furthermore, we introduce a public black-box tracing mechanism that can accurately identify malicious leakers without requiring decryption. In our framework, each client generates their own public/secret key pair locally and completes registration through a transparent key curator (KC). The KC only provides a public service and is not entrusted with any secret information, thus effectively mitigating key escrow risks. To instantiate the framework, we construct a concrete scheme based on bilinear pairings and prove that it achieves very selective simulation-based security and public black-box traceability in the standard model. Experimental results show that for the message vector dimension n up to 100 and the client registration number L

reaching 100, the total communication overhead of our TRFE-UIP scheme exhibits a near-linear growth with n and L . Moreover, its total computational cost is reduced by 96.4% compared with that of the current mainstream scheme. Therefore, our scheme is highly suitable for Web application scenarios involving large-scale user access and high-dimensional vector interactions.

Index Terms—Registered functional encryption, selective simulation-based security, traceability, unbounded inner product.

I. INTRODUCTION

WITH the rapid development of cloud computing, web service platforms (e.g., online healthcare, precision marketing, social networks) have become critical hubs for cross-domain data sharing and collaborative computing. The core services of such platforms include similarity search, recommendation systems, and paid queries, all of which rely on numerical calculations of high-dimensional user data. Specifically, the inner product operation serves as the core technical foundation for supporting essential functions such as similarity analysis and preference matching. Its accuracy and efficiency directly determine the service quality and user experience of Web service platforms. However, all these Web services involve large amounts of high-dimensional sensitive user data, including personal attributes, behavioral trajectories, preference vectors, etc. Such data are often acquired and processed by platform servers, which are generally not fully trusted. These servers may illegally parse the data and even disclose or sell sensitive information to unauthorized entities, which severely hinders the healthy development of Web service platforms. Therefore, enabling effective inner product computation on high-dimensional data while guaranteeing user data privacy has emerged as a critical research challenge.

Functional encryption (FE) [1], as a novel cryptographic paradigm, offers the powerful ciphertext computing capability that is perfectly applied to the aforementioned Web application scenarios. In FE, a data owner encrypts a message x using a master public key mpk , obtaining a ciphertext ct . Any entity with a functional key sk_f can compute $f(x)$ on ct directly without learning x . However, the classical FE relies on a single trusted authority (TA) that generates a master secret key msk and distributes all functional keys sk_f , which entails an inherent key-escrow risk. But this is in serious conflict with the architecture of Web platforms, which features no central trust and cross-domain collaboration.

Received 16 January 2026; revised 25 May 2026; accepted 18 July 2026. Date of publication 23 July 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundations of China under Grant 12441101 and Grant 62372285, in part by the Science Foundation of Zhejiang Sci-Tech University under Grant 26222227-Y, in part by the Program of Zhejiang Key Laboratory of Digital Fashion and Data Governance under Grant 2024E10049, in part by the National Key Research and Development Program of China under Grant 2025YFA1017100 and Grant 2023YFB2703700, in part by the Fundamental Research Funds for the Central under Grant 2042026kf0012 and Grant 2042024kf1013, in part by the Program for Leading Innovative Research Team under Grant 2023R01001, and in part by the Shuguang Program of Shanghai Education Development Foundation and under Grant 25SG49. (Corresponding author: Jianting Ning.)

Jinmei Tian and Weijing You are with the School of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China (e-mail: tianjinmei@163.com; youweijing@fjnu.edu.cn).

Jianting Ning is with the School of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China, also with the College of Information Science and Engineering, Zhejiang Sci-Tech University, Hangzhou 310018, China, and also with the Faculty of Data Science, City University of Macau, Macau, SAR 999078, China (e-mail: jtning88@gmail.com).

Jian Shen is with the College of Information Science and Engineering, Zhejiang Sci-Tech University, Hangzhou 310018, China (e-mail: s_shenjian@126.com).

Kai Zhang is with the Faculty of Artificial Intelligence, Shanghai University of Electric Power, Shanghai 201399, China (e-mail: kzhang@shiep.edu.cn).

Lefeng Zhang is with the Faculty of Data Science, City University of Macau, Macau, SAR 999078, China (e-mail: lfzhang@cityu.edu.mo).

Digital Object Identifier 10.1109/TDSC.2026.3716326

To remove the key-escrow threat, registered functional encryption (RFE) was proposed by Francati et al. [2], which successfully meets the deployment requirements of the Web platform. RFE introduces a transparent key curator (KC) to replace TA, but KC is solely responsible for aggregating user public keys and publicly updating the system parameters without any secret information. Each user independently generates their key pair (pk, sk) , and then submits pk associated with a designated function f to the KC for registration. After aggregating all public keys, KC outputs a master public key mpk for each registered user with helper decryption key hsk . Then, a data owner produces a ciphertext ct by encrypting a message x with the current mpk . Subsequently, a registered user can decrypt ct using (sk, hsk) to recover $f(x)$. In addition, as the user scale of Web service platforms changes dynamically, the registration of new users triggers a global update of the system master public key mpk . Thus, registered users must periodically retrieve the latest hsk to maintain normal ciphertext decryption and computation capabilities. This key update mechanism must satisfy public verifiability and efficient computability to support the elastic scalability requirements of large-scale Web service platforms.

Since FE and RFE were proposed, their potential in privacy protection and flexible computation has attracted widespread attention and given rise to a series of secure and efficient practical schemes [3], [4], [5], [6], [7] that provide important technical support for privacy-preserving computation in Web service scenarios.

A. Related Work

The concept of functional encryption (FE) was first introduced by Sahai and Waters [1] as an extension of traditional public-key encryption. Boneh et al. [8] further refined the formal definition of FE and proposed two main security notions: indistinguishability (IND) security and simulation-based (SIM) security. The latter requires that an adversary, given only the function output, cannot distinguish between real and ideal environments, thereby providing a stronger privacy guarantee. [9] extended FE to multi-input scenarios, promoting its applications in secure multi-party computation. Since then, a multitude of FE schemes have been proposed [10], [11], [12], [13]. However, these schemes predefine the size or structure of messages, which limits their dynamic extensibility.

To support dynamic environments and variable input lengths, unbounded FE has become a research hotspot. Garg et al. [14] first proposed an FE scheme that supports arbitrary circuits based on indistinguishable obfuscation ($i\mathcal{O}$) and fully homomorphic encryption. In [15], unbounded multi-input FE achieves the selective IND security via public-coin differing-inputs obfuscation. Tomida et al. [16] proposed an FE scheme for unbounded inner product without $i\mathcal{O}$. To deal with vectors of different lengths, they divide the high-dimensional vector into multiple one-dimensional sub-vectors, and append random components (summing to zero) to the function vector and a constant 1 to the plaintext vector until both have the same length. Therefore, the

calculation of an unbounded inner product is reduced to fixed-dimensional inner product. Later, unbounded FE schemes [17], [18], [19], [20] have been extensively studied.

In FE systems, TA generates functional keys for all users. Users might secretly share their functional keys or even collude with others to obtain decryption capabilities beyond their authorization, while the TA cannot identify the exact source of key leakage. Do et al. [21] first introduced traitor tracing into IPFE systems, and proposed an IPFE scheme supporting black-box traceability. However, this scheme requires maintaining a stateful identity-to-index mapping table and only achieves single-challenge security and private tracing. Luo et al. [22] presented a general construction of traceable and retractable IPFE, and their scheme implemented adaptive security and public black-box traceability under standard assumptions. Subsequently, Dutta et al. [23] introduced the black-box tracing mechanism to identity-based IPFE and multi-client IPFE, respectively. Ref. [24] proposed a traceable IPFE scheme with white-box traceability, which achieves user identity tracing. However, all the schemes rely on TA to generate system parameters and user keys, which leads to the risk of key escrow.

To solve the key escrow issue of FE, Francati et al. [2] introduced the idea of registration-based encryption into FE and proposed the concept of RFE. They also constructed a registered IPFE (RIPFE) scheme over bilinear groups, but its security proof relies on a general group model. In 2024, Zhu et al. [25] first implemented an adaptively IND-secure RIPFE scheme under the standard model, and also constructed two RFE schemes that respectively support inner product and quadratic function which both achieve very selective SIM (Sel*-SIM) security. In addition, Hohenberger et al. [26] constructed a registered attribute-based encryption (RABE) scheme with arbitrary access policies based on $i\mathcal{O}$. Later, Francati et al. [2] also proposed an RFE that supports arbitrary function spaces, but still relies on $i\mathcal{O}$. Zhang et al. [27] presented an RFE scheme for any polynomial size circuit, achieving adaptive SIM security under the bounded collusion model. However, the construction is still based on black-box primitives such as obfuscated circuits. Du et al. [28] further removed the black-box tool and first constructed an RIPFE scheme for unbounded inner product, achieving adaptive IND security under the k -Lin assumption. Moreover, Branco et al. [29] introduced traitor traceability into RFE, and proposed adaptively IND-secure traceable RFE for quadratic function in the generic-group model and selectively IND-secure traceable RFE for inner product in the standard model.

B. Motivations and Contributions

At present, FE has evolved from fixed-dimensional single-input scenarios to the RFE stage with unbounded dynamic input, and gradually integrated with traitor tracing to achieve key accountability, which becomes a key primitive for addressing privacy-preserving computation and access control in Web services. However, the following critical challenges remain:

- 1) Most FE schemes can only handle inner product or quadratic function of a predefined dimension, which fail

TABLE I
NOTATIONS AND DESCRIPTIONS

Notations	Descriptions
λ	A security parameter
p	A large prime number
$\mathbb{N}$	The set of natural numbers
$\mathbb{R}$	The limited field
$\mathbb{Z}_p$	The prime field of order p
$\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$	Three cyclic groups with order p
$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$	A bilinear map
g_1 or g_2	A generator of $\mathbb{G}_1$ or $\mathbb{G}_2$
$S, S $	A finite set and its size
$\mathcal{L}, \mathcal{L} $	An ordered list or array and its size
$\mathcal{L}[i]$	the i -th element of $\mathcal{L}$
$\approx_c$	Computationally indistinguishable
lowercase (e.g. $\mathbf{a}$)	The row vectors
uppercase (e.g. $\mathbf{A}$)	The matrices
$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}\mathbf{y}^\top$	The inner product of row vector $\mathbf{x}$ and $\mathbf{y}$
$[\mathbf{A}]_s$	$g_s^{a_{ij}}$ where $\mathbf{A} = (a_{ij})$ and $s \in \{1, 2, T\}$
$\mathbf{A} \otimes \mathbf{B}$	Kronecker product for $\mathbf{A} \in \mathbb{R}^{\ell \times m}$, $\mathbf{B} \in \mathbb{R}^{n \times k}$
$x \leftarrow X$	x is selected from X randomly
$\text{AL}(x) \rightarrow y$	x is computed by running the algorithm $\text{AL}(x)$

to accommodate high-dimensional and dynamically expanding data. Even the few existing unbounded RIPFE schemes rely on black-box primitives such as $i\mathcal{O}$, making them difficult to directly apply to naturally variable-length high-dimensional data streams in Web services.

- 2) Within FE research, especially RFE, there is a widespread lack of effective accountability mechanisms. Since existing schemes [1], [2], [3], [4], [5], [6], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20] do not embed unique user identifiers into the functional key generation process which is exactly the same for all participating entities (users and TA/KC), it is impossible to distinguish the originally authorized user based solely on the key. Once functional keys are maliciously shared or leaked, the source of leakage can be easily concealed. TA/KC cannot achieve precise accountability, directly threatening the security and compliance of access control for Web services.
- 3) The security models of FE and RFE schemes are mostly limited to IND security. However, Web services are exposed to adversarial scenarios such as adaptive attacks, collusion attacks, and key forgery attacks in the public network environment. The IND security cannot guarantee the privacy of ciphertext and functions, making it difficult to provide privacy protection for Web services.

In this work, we propose a traceable registered functional encryption for unbounded inner product (TRFE-UIP) scheme without $i\mathcal{O}$ and prove that it satisfies Sel*-SIM security and public black-box traceability under the standard model. More concretely, our contributions are summarized as follows:

- 1) We first formalize a comprehensive definitional framework for TRFE-UIP. On this basis, we define the formal security models of Sel*-SIM security and public black-box traceability, thus establishing a security and traceability benchmark for subsequent constructions.
- 2) By instantiating the framework, we propose a concrete TRFE-UIP scheme. It realizes unbounded inner product without $i\mathcal{O}$ and introduces a public black-box tracing

mechanism for accurately identifying malicious key distributors without decryption. Under the standard model, our scheme is proved to achieve Sel*-SIM security and public black-box traceability.

- 3) Theoretical analysis and experimental evaluation show that under consistent security parameter settings, the sizes of master public key, ciphertext, and encryption/decryption keys in our scheme scale linearly with both the message vector dimension n and the number L of registered clients. This effectively avoids the exponential overhead redundancy and the explosive growth of $(n - L)$ coupled terms observed in existing IPFE schemes. Compared to the scheme in [27], the total computational cost of our scheme is reduced by 96.4%. Furthermore, our scheme outperforms the scheme of [21] in high-dimensional settings. These results confirm that our proposed scheme achieves superior overall performance to the existing single-function or IPFE schemes, laying a solid foundation for practical deployment.

II. PRELIMINARIES

In this section, we begin by presenting the notations adopted throughout this paper. Then, we briefly review the prime-order bilinear groups, complexity assumptions and some cryptographic tools used by our TRFE-UIP scheme.

A. Notations

The notations used in the paper are shown in Table I.

B. Bilinear Group and Complexity Assumptions

Let λ be a security parameter and p be a λ -bit prime number, an efficient $\mathcal{G}$ is a generator of a asymmetric bilinear group that takes as input a security parameter 1^λ and outputs $\mathbb{G} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, g_1, g_2, e)$, where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are three cyclic groups with order p , $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a non-degenerate bilinear map.

The security of our TRFE-UIP scheme is based on the following complexity assumption.

The (k, ℓ, d) -Matrix Diffie-Hellman $((k, \ell, d)$ -MDDH) problem [30] is defined as follows: Given $k, \ell, d \in \mathbb{N}$, $\mathbb{G} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \mathcal{G}(1^\lambda)$ and $\mathbf{M} \in \mathbb{Z}_p^{\ell \times k}$, compute $\mathbf{SM}$ for randomly chosen $\mathbf{S} \in \mathbb{Z}_p^{d \times k}$ and determine whether $\mathbf{SM}$ is equal to $\mathbf{U} \in \mathbb{Z}_p^{d \times k}$ holds true. An algorithm $\mathcal{A}$ has advantage ϵ in solving the (k, ℓ, d) -MDDH problem in $\mathbb{G}$ if

$$\text{Adv}_{\mathcal{A}, s, k, \ell, d}^{\text{MDDH}}(\lambda) = \left| \Pr \left[\mathcal{A}(\mathbb{G}, [\mathbf{M}]_s, [\mathbf{SM}]_s) = 1 \right] - \Pr \left[\mathcal{A}(\mathbb{G}, [\mathbf{M}]_s, [\mathbf{U}]_s) = 1 \right] \right| \geq \epsilon$$

where the probability is over the random choices of $\mathbf{S} \in \mathbb{Z}_p^{d \times k}$ and $\mathbf{U} \in \mathbb{Z}_p^{d \times k}$.

We demonstrate that this assumption follows directly from the k -Lin. From [30], we know that the DDH (or SXDH) assumption is an instance of the k -Lin assumption over symmetric group (or

asymmetric group) when $k = 1$. When $k = 2$, the assumption corresponds to the BDDH assumption on the bilinear group.

Definition 1: $((k, \ell, d)$ -MDDH Assumption). The $((k, \ell, d)$ -MDDH assumption means that there is no probabilistic polynomial-time (PPT) algorithm $\mathcal{A}$ that can solve the $((k, \ell, d)$ -MDDH problem with non-negligible advantage ϵ .

C. Cryptographic Tools

Definition 2: (Limited-Norm Inner Product) [16]. The function family $\mathbb{F}$ consists of functions $f_{\mathcal{I}, \mathbf{y}}^{X, Y} : \mathbb{R}^m \rightarrow \mathbb{R}$ where $X, Y \in \mathbb{N}$, $m \in \mathbb{N}$, $\mathcal{I} \subseteq [m]$ and $\mathbf{y} = (y_i)_{i \in \mathcal{I}} \in \mathbb{Z}^{\mathcal{I}}$ such that $\|\mathbf{y}\|_{\infty} \leq Y$, and $\mathbb{R}$ is either $\mathbb{Z}$ or $\mathbb{Z}_p$ for some integer p . Let $\mathcal{X} := \{X_{\lambda}\}_{\lambda \in \mathbb{N}}$, $\mathcal{Y} := \{Y_{\lambda}\}_{\lambda \in \mathbb{N}}$ be ensembles of norm-limit. For every $\mathbf{x} = (x_1, \dots, x_m) \in \mathbb{Z}^m$ such that $\|\mathbf{x}\|_{\infty} \leq X$ and $X \in \mathbb{N}$, the inner product function works as:

$$f_{\mathcal{I}, \mathbf{y}}^{X, Y}(x_1, \dots, x_m) = \sum_{i \in \mathcal{I}} x_i y_i.$$

Definition 3: A quasi-adaptive non-interactive zero-knowledge argument (QA-NIZK) for linear space over bilinear group $\mathbb{G}$ [31], [32] consists of four efficient algorithms (LGen, LPrv, LVer, LSim) that work as follows:

- 1) LGen($1^\lambda, 1^n, 1^m, 1^\ell, [\mathbf{M}]_1$) $\rightarrow$ (crs, td): On input the security parameter 1^λ , language parameter $1^n, 1^m, 1^\ell$, and a matrix $[\mathbf{M}]_1 \in \mathbb{G}_1^{n \times m}$ defining a linear space, it outputs common reference string crs and trapdoor td.
- 2) LPrv(crs, $[\mathbf{Y}]_1, \mathbf{X}$) $\rightarrow \pi$: On input crs, $[\mathbf{Y}]_1 \in \mathbb{G}_1^{n \times \ell}$ and $\mathbf{X} \in \mathbb{Z}_p^{m \times \ell}$ such that $\mathbf{Y} = \mathbf{M}\mathbf{X}$, it outputs a proof π .
- 3) LVer(crs, $[\mathbf{Y}]_1, \pi$) $\rightarrow 0/1$: On input crs, $[\mathbf{Y}]_1$ and π , it outputs a bit showing the validity of π .
- 4) LSim(crs, td, $[\mathbf{Y}]_1$) $\rightarrow \tilde{\pi}$: On input crs, td, $[\mathbf{Y}]_1$, it outputs a simulated proof $\tilde{\pi}$.

Perfect Completeness: For all $\lambda, \mathbf{M}$, and all $\mathbf{X}, \mathbf{Y}$ such that $\mathbf{Y} = \mathbf{M}\mathbf{X}$:

$$\Pr \left[\text{LVer}(\text{crs}, [\mathbf{Y}]_1, \pi) = 1 \mid \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \text{LGen}(1^\lambda, 1^n, \\ 1^m, 1^\ell, [\mathbf{M}]_1) \\ \pi \leftarrow \text{LPrv}(\text{crs}, [\mathbf{Y}]_1, \mathbf{X}) \end{array} \right] = 1.$$

Perfect Zero-knowledge: For all $\lambda, \mathbf{M}$, (crs, td) $\leftarrow$ LGen($1^\lambda, 1^n, 1^m, 1^\ell, [\mathbf{M}]_1$), and all $\mathbf{X}, \mathbf{Y}$ such that $\mathbf{Y} = \mathbf{M}\mathbf{X}$:

$$\text{LPrv}(\text{crs}, [\mathbf{Y}]_1, \mathbf{X}) \equiv \text{LSim}(\text{crs}, \text{td}, [\mathbf{Y}]_1).$$

Unbounded Simulation Soundness: For all adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} ([\mathbf{Y}^*]_1, \pi) \notin Q \\ \wedge \mathbf{Y}^* \notin \text{span}(\mathbf{M}) \\ \wedge \text{LVer}(\text{crs}, [\mathbf{Y}^*]_1, \\ \pi^*) = 1 \end{array} \mid \begin{array}{l} \mathbf{M} \in \mathbb{Z}_p^{n \times m} \\ (\text{crs}, \text{td}) \leftarrow \text{LGen}(1^\lambda, 1^n, 1^m, 1^\ell, \\ [\mathbf{M}]_1) \\ ([\mathbf{Y}^*]_1, \pi^*) \leftarrow \mathcal{A}^{\text{LSim}(\text{crs}, \text{td}, \cdot)}(1^\lambda, \\ \text{crs}, \mathbf{M}) \end{array} \right]$$

is negligible in λ , where Q records all queries to LSim(crs, td, $\cdot$) along with response. $\text{Adv}_{\mathcal{A}, n, m, \ell}^{\text{USS}}(\lambda)$ is denoted as the advantage function. Note that the definition is stronger in the sense that the adversary is given $\mathbf{M}$ instead of $[\mathbf{M}]_1$.

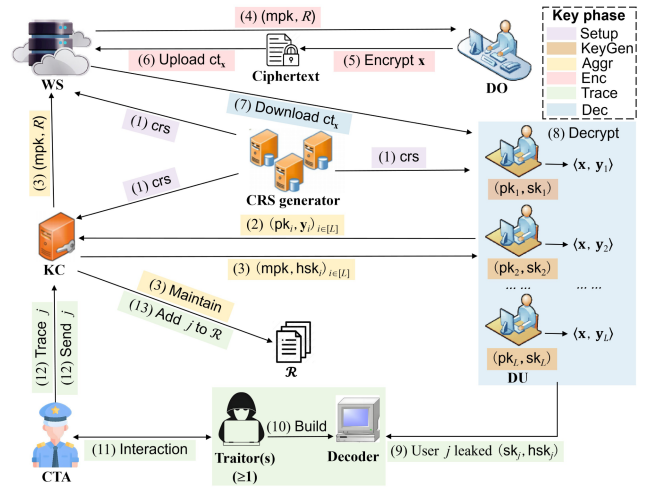


Fig. 1. The system model of TRFE-UIP.

III. FORMAL DEFINITIONS OF TRFE-UIP

In this section, we formalize the system model, scheme and security definitions of our TRFE-UIP scheme.

A. System Model

As shown in Fig. 1, five entities are involved in a TRFE-UIP scheme, including the common reference string (CRS) generator, a key curator (KC), two types of client(data owner (DO) and data user (DU)), a web server (WS), and a central tracing authority (CTA). In the TRFE-UIP system, the CRS generator as a trusted third party initializes the entire system, generates and publishes a common reference string crs based on security parameters. Here, the client i (either DO or DU) locally generates its own key pair (pk_i, sk_i) . Then, it submits a function vector $\mathbf{y}_i$ and pk_i to the KC for registration. During the registration phase, KC generates a master public key mpk and issues a helper decryption key hsk_i to each client i without retaining any secret information. Furthermore, KC undertakes the user revocation and keeps a revocation list $\mathcal{R}$.

To achieve secure data sharing, DO encrypts its data vector $\mathbf{x}$ and uploads the resulting ciphertext $ct_{\mathbf{x}}$ to WS. Meanwhile, any revoked user cannot perform decryption operations. As long as a DU is not revoked, it can download $ct_{\mathbf{x}}$ from the WS and then decrypt it using (sk_i, hsk_i) . Notably, different DUs obtain distinct decryption results, specifically $\langle \mathbf{x}, \mathbf{y}_i \rangle$. In addition, WS is considered honest-but-curious, responsible for storing ciphertexts and some public information, such as mpk and $\mathcal{R}$. If a certain client (e.g., client j) leaks its secret key pair (sk_j, hsk_j) to unauthorized parties, it is deemed a “traitor”. One or more such clients may collectively form a decryption black box, referred to as the “Decoder”. When such a key leakage incident occurs, the CTA can interact with the decoder and run a trace algorithm to identify the traitor. When receiving the traitor’s index j , KC revokes and adds j to $\mathcal{R}$, thereby preventing its further decryption.

B. Scheme and Security Definitions

Definition 4: A traceable registered functional encryption for unbounded inner product (TRFE-UIP) scheme is formally defined by using a set of efficient algorithms (Setup, KeyGen, IsValid, Aggr, Enc, Trace^D, Dec) as follows:

- $\text{Setup}(1^\lambda, 1^L) \rightarrow \text{crs}$: The setup algorithm is executed by the CRS generator. It takes security parameter λ and the number of users L as input, outputs a common reference string crs .
- $\text{KeyGen}(\text{crs}, i) \rightarrow (\text{pk}_i, \text{sk}_i)$: The key generation algorithm is performed by each client. It takes the common reference string crs and a user $i \in [L]$ as input, and outputs a key pair $(\text{pk}_i, \text{sk}_i)$ for client i .
- $\text{IsValid}(\text{crs}, i, \text{pk}_i) \rightarrow b$: The algorithm is run by KC or others to verify whether any public key is valid. It takes the common reference string crs , a user $i \in [L]$ and public key pk_i as input, and outputs a bit $b \in \{0, 1\}$.
- $\text{Aggr}(\text{crs}, \mathcal{I}, \{\text{pk}_i, \mathbf{y}_i\}_{i \in [L]}) \rightarrow (\text{mpk}, \{\text{hsk}_{\mathbf{y}_j}\}_{j \in [L]})$: The aggregation algorithm is executed by KC. It takes the common reference string crs , a non-empty index set $\mathcal{I} \subseteq [r_i]$ where $r_i = r_i(\lambda)$ is any polynomial, a set of L public keys pk_i with vector $\mathbf{y}_i = (y_{i,u})_{u \in [r_i]} \in [\mathbb{Z}_p^{r_i}]$ as input, and outputs a master public key mpk and a series of helper decryption keys $\text{hsk}_{\mathbf{y}_1}, \dots, \text{hsk}_{\mathbf{y}_L}$.
- $\text{Enc}(\text{mpk}, \mathbf{x}, \mathcal{R}) \rightarrow \text{ct}_{\mathbf{x}}$: The encryption algorithm is performed by DO. It takes the master public key mpk , a message vector $\mathbf{x} = (x_1, \dots, x_m) \in \mathbb{Z}_p^m$ where $m = m(\lambda)$ is any polynomial, and a revocation set $\mathcal{R}$ ($|\mathcal{R}| \leq L$) which contains the index of every revoked user as input, and outputs a ciphertext $\text{ct}_{\mathbf{x}}$.
- $\text{Trace}^D(\text{mpk}, \mathcal{R}, \mu(\cdot), \mathbf{x}_0^*, \mathbf{x}_1^*) \rightarrow \mathcal{L} \subseteq [L]$: CTA runs the trace algorithm by interacting with a decoder D . It takes the master public key mpk , revocation set $\mathcal{R}$, two different message vectors $\mathbf{x}_0^*, \mathbf{x}_1^* \in \mathbb{Z}_p^m$ which are obtained from the decoder D , and a function $\mu(\cdot)$ in λ representing the probability that D can distinguish between the ciphertexts of $\mathbf{x}_0^*$ and $\mathbf{x}_1^*$ as input, it has oracle access to a decoder D and outputs an index set $\mathcal{L} \subseteq [L]$ of malicious user(s).
- $\text{Dec}(\text{sk}_i^*, \text{hsk}_{\mathbf{y}_{i^*}}, \text{ct}_{\mathbf{x}}) \rightarrow \langle \mathbf{x}, \mathbf{y}_{i^*} \rangle \cup \perp$: The decryption algorithm is executed by DU. It takes a secret key sk_i^* , a helper secret key $\text{hsk}_{\mathbf{y}_{i^*}}$ for vector $\mathbf{y}_{i^*} = (y_{i^*,u})_{u \in \mathcal{I}} \in [\mathbb{Z}^{\mathcal{I}}]$ and a ciphertext $\text{ct}_{\mathbf{x}}$ for message vector $\mathbf{x} = (x_1, \dots, x_m) \in \mathbb{Z}^m$ as input, and outputs $\langle \mathbf{x}, \mathbf{y}_{i^*} \rangle$, or $\perp$ indicating decryption failure if $\mathcal{I} \not\subseteq [m]$ or $i^* \notin \mathcal{R}$.

Definition 5: (Completeness). A TRFE-UIP scheme is complete if for any λ , $L \in \mathbb{N}$ and $i \in [L]$, it holds that

$$\Pr \left[\begin{array}{l} \text{IsValid}(\text{crs}, i, \text{pk}_i) \\ = 1 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^L) \\ (\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{crs}, i) \end{array} \right] = 1.$$

Definition 6: (Correctness). For any λ , $L \in \mathbb{N}$, $i \in [L]$, all $\text{crs} \leftarrow \text{Setup}(1^\lambda, 1^L)$, $(\text{pk}_{i^*}, \text{sk}_{i^*}) \leftarrow \text{KeyGen}(\text{crs}, i^*)$, and all collections of public keys $\{\text{pk}_i\}_{i \in [L] \setminus \{i^*\}}$ such that $\text{IsValid}(\text{crs}, i, \text{pk}_i) = 1$, all $\mathbf{x} \in \mathbb{Z}^m$ where $m = m(\lambda)$ is any polynomial and $\|\mathbf{x}\|_\infty \leq X_\lambda$, index set $\mathcal{I} \subseteq [r_i]$ where $r_i =$

$r_i(\lambda)$ is any polynomial, and all $\mathbf{y}_i \in \mathbb{Z}^{\mathcal{I}}$ for $i \in [L]$ such that $\|\mathbf{y}_i\|_\infty \leq Y_\lambda$, a TRFE-UIP scheme is correct if $\mathcal{I} \subseteq [m]$ and it holds that

$$\Pr \left[\begin{array}{l} i^* \notin \mathcal{R} \wedge \\ d = \langle \mathbf{x}, \mathbf{y}_{i^*} \rangle \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^L) \\ (\text{pk}_{i^*}, \text{sk}_{i^*}) \leftarrow \text{KeyGen}(\text{crs}, i^*) \\ (\text{mpk}, \{\text{hsk}_{\mathbf{y}_j}\}_{j \in [L]}) \leftarrow \text{Aggr}(\text{crs}, \\ \mathcal{I}, \{\text{pk}_i, \mathbf{y}_i\}_{i \in [L]}) \\ \text{ct}_{\mathbf{x}} \leftarrow \text{Enc}(\text{mpk}, \mathbf{x}, \mathcal{R}) \\ d \leftarrow \text{Dec}(\text{sk}_{i^*}, \text{hsk}_{\mathbf{y}_{i^*}}, \text{ct}_{\mathbf{x}}) \end{array} \right] = 1.$$

Remark 1: The above correctness guarantees expected decryption behavior for all legitimate non-revoked users. For any non-revoked user $i^* \notin \mathcal{R}$, if function vector $\mathbf{y}_{i^*}$ is well-formed and its effective dimension (index set $\mathcal{I}$) is within that of message vector $\mathbf{x}$, the Dec algorithm outputs $\langle \mathbf{x}, \mathbf{y}_{i^*} \rangle$. Otherwise, decryption fails when $i^* \in \mathcal{R}$ or $\mathcal{I} \not\subseteq [m]$.

Definition 7: (Very Selective SIM-Security). A TRFE-UIP scheme is very selective SIM-secure if there exists an efficient simulator $\text{Sim} = (\text{Setup}, \text{KeyGen}, \text{Enc})$ such that for any PPT adversary $\mathcal{A}_1$, the outputs of the following two experiments are computationally indistinguishable:

Experiment Real _{$\mathcal{A}_1^{\text{TRFE-UIP}}$} (1^λ):

$$\begin{aligned} L, \mathbf{x}^*, \{\mathbf{y}_i^*\}_{i \in [L]}, \mathcal{M}^*, \mathcal{C}^* &\leftarrow \mathcal{A}_1(1^\lambda); \\ \text{crs} &\leftarrow \text{Setup}(1^\lambda, 1^L); \\ \{\text{pk}_i^*\}_{i \in [L]} &\leftarrow \mathcal{A}_1^{\text{OGen}(\cdot), \text{OCor}(\cdot, \cdot)}(\text{crs}); \\ (\text{mpk}, \cdot) &\leftarrow \text{Aggr}(\text{crs}, \mathcal{I}, \{\text{pk}_i^*, \mathbf{y}_i^*\}_{i \in [L]}); \\ \text{ct}_{\mathbf{x}^*} &\leftarrow \text{Enc}(\text{mpk}, \mathbf{x}^*, \mathcal{R}); \\ \alpha &\leftarrow \mathcal{A}_1^{\text{OCor}(\cdot, \cdot)}(\text{ct}_{\mathbf{x}^*}); \end{aligned}$$

Output $(\mathbf{x}^*, \{\mathbf{y}_i^*\}_{i \in [L]}, \alpha)$.

$\approx_c$
Experiment Ideal _{$\mathcal{A}_1^{\text{TRFE-UIP}}$} (1^λ):

$$\begin{aligned} L, \mathbf{x}^*, \{\mathbf{y}_i^*\}_{i \in [L]}, \mathcal{M}^*, \mathcal{C}^* &\leftarrow \mathcal{A}_1(1^\lambda); \\ \{\widetilde{\text{crs}}, \text{td}\} &\leftarrow \widetilde{\text{Setup}}(1^\lambda, 1^L; \{\mathbf{y}_i^*\}_{i \in [L]}, \{\langle \mathbf{x}^*, \mathbf{y}_i^* \rangle\}_{i \in \mathcal{M}^* \cup \mathcal{C}^*}); \\ \{\text{pk}_i^*\}_{i \in [L]} &\leftarrow \mathcal{A}_1^{\text{OGen}(\cdot), \text{OCor}(\cdot, \cdot)}(\widetilde{\text{crs}}); \\ \widetilde{\text{ct}}_{\mathbf{x}^*} &\leftarrow \widetilde{\text{Enc}}((\text{pk}_1^*, \dots, \text{pk}_{L'}^*), \mathbf{x}^*, \mathcal{R}; \text{td}); \\ \alpha &\leftarrow \mathcal{A}_1^{\text{OCor}(\cdot, \cdot)}(\widetilde{\text{ct}}_{\mathbf{x}^*}); \end{aligned}$$

Output $(\mathbf{x}^*, \{\mathbf{y}_i^*\}_{i \in [L]}, \alpha)$.

where message vector $\mathbf{x}^* \in \mathbb{Z}^m$ where $m = m(\lambda)$ is any polynomial, index set $\mathcal{I} \subseteq [r_i]$ where $r_i = r_i(\lambda)$ is any polynomial, and all function vector $\mathbf{y}_i = (y_{i,u})_{u \in \mathcal{I}} \in \mathbb{Z}^{\mathcal{I}}$ for $i \in [L]$ such that $\|\mathbf{y}_i\|_\infty \leq Y_\lambda$ is any polynomial. In addition, $\mathcal{M}^*, \mathcal{C}^* \subseteq [L]$ are the sets of malicious and corrupted users, respectively. If $j \in \mathcal{M}^*$, revocation set $\mathcal{R} = \mathcal{R} \cup \{j\}$. And the oracles $\text{OGen}(\cdot)$ and $\text{OCor}(\cdot, \cdot)$ works as follows with the initial setting $\mathcal{C} = \emptyset$ and $\mathcal{D}_i = \emptyset$ for $i \in [L]$:

- $\text{OGen}(i)$: run $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{crs}, i)$, set $\mathcal{D}_i[\text{pk}_i] = \text{sk}_i$ and return pk_i .
- $\text{OCor}(i, \text{pk}_i)$: return $\mathcal{D}_i[\text{pk}_i]$, update $\mathcal{C} = \mathcal{C} \cup \{(i, \text{pk}_i)\}$.

In the ideal world, the oracles are analogous to that in the real world, except that $\text{OGen}(\cdot)$ invokes $\widehat{\text{KeyGen}}$ instead of KeyGen and the following restrictions:

- $i \in \mathcal{M}^* \Rightarrow \mathcal{D}_i[\text{pk}_i^*] = \perp \wedge \text{Ver}(\text{crs}, i, \text{pk}_i^*) = 1$.
- $i \in \mathcal{C}^* \Rightarrow (i, \text{pk}_i^*) \in \mathcal{C}$.
- $i \in [L] \setminus (\mathcal{M}^* \cup \mathcal{C}^*) \Rightarrow \mathcal{D}_i[\text{pk}_i^*] \neq \perp \wedge (i, \text{pk}_i^*) \notin \mathcal{C}$.

Similarly, there is no need to give $\text{mpk}, \text{hsk}_{\mathbf{y}_1}^*, \dots, \text{hsk}_{\mathbf{y}_L}^*$ to $\mathcal{A}_1$ explicitly in real game (or explicitly in simulation game) and consider post-challenge queries.

Remark 2: In our security model, the CRS generator is trusted only once upon system initialization, while KC does not hold any client long-term private keys. Like most RFE schemes [26], [27], [28], [29], we focus on the minimal trust assumptions and do not yet consider the impact when the crs is maliciously constructed or when KC is malicious. We will discuss the impact and corresponding solutions in Appendix B (available online).

Definition 8: (μ -useful Black-box Distinguisher [22]). For a non-negligible function $\mu(\cdot)$ in λ and a PPT algorithm $\mathcal{P}$, we say that a black-box distinguisher D is μ -useful, if we have

$$\left| \Pr \left[\begin{array}{l} D(\text{ct}_{\mathbf{x}^*}^b) \\ = b \\ \text{s.t. } \langle \mathbf{x}_0^*, \mathbf{y}_i \rangle \neq \langle \mathbf{x}_1^*, \mathbf{y}_i \rangle \\ b \leftarrow \{0, 1\} \\ \text{ct}_{\mathbf{x}^*}^b \leftarrow \text{Enc}(\text{mpk}, \mathbf{x}_b^*, \mathcal{R}) \end{array} \right] - \frac{1}{2} \right| \geq \mu(\lambda),$$

where $\text{crs} \leftarrow \text{Setup}(1^\lambda, 1^L)$, $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{crs}, i)$, $(\text{mpk}, \{\text{hsk}_{\mathbf{y}_i}\}_{i \in [L]}) \leftarrow \text{Aggr}(\text{crs}, \mathcal{I}, (\text{pk}_i, \mathbf{y}_i)_{i \in [L]})$ and $\mathcal{R}_j^* = j \in [L]$. We convert the inequality into its equivalent form:

$$\left| \Pr \left[\begin{array}{l} D(\text{ct}_{\mathbf{x}^*}^0) \\ = 0 \\ \text{s.t. } \langle \mathbf{x}_0^*, \mathbf{y}_i \rangle \neq \langle \mathbf{x}_1^*, \mathbf{y}_i \rangle \\ \text{ct}_{\mathbf{x}^*}^0 \leftarrow \text{Enc}(\text{mpk}, \mathbf{x}_0^*, \mathcal{R}) \end{array} \right] - \Pr \left[\begin{array}{l} D(\text{ct}_{\mathbf{x}^*}^1) \\ = 0 \\ \text{s.t. } \langle \mathbf{x}_0^*, \mathbf{y}_i \rangle \neq \langle \mathbf{x}_1^*, \mathbf{y}_i \rangle \\ \text{ct}_{\mathbf{x}^*}^1 \leftarrow \text{Enc}(\text{mpk}, \mathbf{x}_1^*, \mathcal{R}) \end{array} \right] \right| \geq 2\mu(\lambda).$$

Definition 9: (*Public black-box traceability*). A TRFE-UIP scheme satisfies public black-box traceability if for any PPT adversary $\mathcal{A}_2$, it holds that

$$\Pr \left[\text{Adv}_{\text{PBBT}, \mathcal{A}_2}^{\text{TRFE-UIP}}(1^\lambda) = 1 \right] \leq \text{negl}(\lambda),$$

where $\text{Adv}_{\text{PBBT}, \mathcal{A}_2}^{\text{TRFE-UIP}}$ is defined for any μ -useful black-box distinguisher D as follows:

$$\text{Adv}_{\text{PBBT}, \mathcal{A}_2}^{\text{TRFE-UIP}}(1^\lambda) :$$

$$L, \mathbf{x}_0^*, \mathbf{x}_1^*, \{\mathbf{y}_i^*\}_{i \in [L]}, \mathcal{M}^*, \mathcal{C}^* \leftarrow \mathcal{A}_2(1^\lambda);$$

$$\text{crs} \leftarrow \text{Setup}(1^\lambda, 1^L);$$

$$(\{\text{pk}_i^*\}_{i \in [L]}, \{\mathbf{r}_i\}_{i \in \mathcal{M}^*}, D) \leftarrow \mathcal{A}_2^{\text{OGen}(\cdot), \text{OCor}(\cdot, \cdot)}(\text{crs});$$

$$[L] \setminus \mathcal{M}^* \subseteq \mathcal{H}, \text{ where } \mathcal{H} \text{ is the set of honest users;}$$

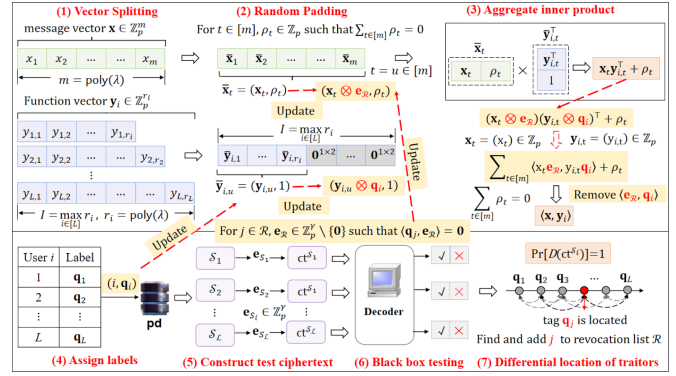


Fig. 2. The technique overview of TRFE-UIP.

$$\forall i \in \mathcal{M}^*, \text{pk}_i \in \text{KeyGen}(\text{crs}, \text{pk}_i^*; \mathbf{r}_i);$$

$$\forall i \in \mathcal{M}^* \cup \mathcal{C}^*, \langle \mathbf{x}_0^*, \mathbf{y}_i^* \rangle = \langle \mathbf{x}_1^*, \mathbf{y}_i^* \rangle;$$

$$(\text{mpk}, \{\text{hsk}_{\mathbf{y}_i^*}\}_{i \in [L]}) \leftarrow \text{Aggr}(\text{crs}, \mathcal{S}, (\text{pk}_i, \mathbf{y}_i^*)_{i \in [L]});$$

$$\mathcal{S}^* \leftarrow \text{Trace}^D(\text{mpk}, \mathcal{R}, \mu(\cdot), \mathbf{x}_0^*, \mathbf{x}_1^*);$$

$$d_1 = (|\Pr[D(\text{ct}_{\mathbf{x}_0^*}) = 0] - \Pr[D(\text{ct}_{\mathbf{x}_1^*}) = 0]| \geq 2\mu(\lambda)),$$

$$\text{where } \text{ct}_{\mathbf{x}^*}^b \leftarrow \text{Enc}(\text{mpk}, \mathbf{x}_b^*, \mathcal{R}) \text{ and } b \leftarrow \{0, 1\};$$

$$d_2 = (\mathcal{S}^* = \emptyset \vee \mathcal{S}^* \not\subseteq \mathcal{S});$$

Output ($d_1 \wedge d_2$).

where $\text{OGen}(\cdot)$ and $\text{OCor}(\cdot, \cdot)$ work the same as those of **Definition 7** except that update $\mathcal{H}[i] = \mathcal{H}[i] \cup \{\text{pk}_i, \text{sk}_i\}$.

IV. THE PROPOSED TRFE-UIP SCHEME

In this section, we will introduce a TRFE-UIP scheme that supports unbounded inner product and traceability feature. Before that, we first provide an overview of the core technologies and then present the concrete construction.

Overview: To support unbounded inner product, we first adopt vector partitioning to split any-length vector into single-element sub-vectors. Then, corresponding random padding is performed on the sub-vectors, and the values at invalid positions are automatically eliminated due to orthogonal conditions. The specific process are shown in Steps (1)–(3) of Fig. 2. To achieve public traceability, each user i is assigned a unique tag $\mathbf{q}_i \in \mathbb{Z}_p^\gamma$ during registration. CTA constructs distinguishable ciphertexts for different user subsets $\mathcal{S}_i$, and locates traitors by observing behavioral changes of the decoder through black-box testing. The detailed process are illustrated in Steps (4)–(7). In addition, KC embeds the tag of user i into the registered function vector, ensuring that any decryption capability derived from a malicious user's key retains a unique tracking tag. KC also stores $(i, \mathbf{q}_i)$ in a public directory pd , enabling any third party to verify the validity of user tags and guaranteeing the transparency of the tracing process.

Our Construction: Let $\Pi_0 = (\text{LGen}, \text{LPrv}, \text{LVer}, \text{LSim})$ be a QA-NIZK scheme for linear space over bilinear groups, our TRFE-UIP scheme works as follows:

Setup($1^\lambda, 1^L$) $\rightarrow$ crs: The algorithm is executed by the CRS generator to initialize the system. On input λ and L , the CRS generator samples $\mathbb{G} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \mathcal{G}(1^\lambda)$ and generates the following:

- 1) Sample $\mathbf{A} \leftarrow \mathbb{Z}_p^{k \times (2k+1)}$, $\mathbf{V}_1 \leftarrow \mathbb{Z}_p^{(2k+1) \times (\gamma+1)}$, $\mathbf{V}_2 \leftarrow \mathbb{Z}_p^{(2k+1) \times (k+1)}$, $\mathbf{v} \leftarrow \mathbb{Z}_p^{1 \times (2k+1)}$;
- 2) Sample $\mathbf{B} \leftarrow \mathbb{Z}_p^{(k+1) \times k}$, $\mathbf{D} \leftarrow \mathbb{Z}_p^{(k+1) \times k}$, $\mathbf{w} \leftarrow \mathbb{Z}_p^{1 \times (k+1)}$;
- 3) For all $i \in [L]$, run following operations:
 - Select $\mathbf{t}_i \leftarrow \mathbb{Z}_p^{1 \times k}$ and for $s \in \{1, 2\}$, set

$$[\mathbf{M}_i]_s = \begin{bmatrix} \mathbf{I}_\gamma & \mathbf{0}_{\gamma \times 1}^\top \\ \mathbf{0}_{(k+1) \times \gamma} & \mathbf{D} \mathbf{t}_i^\top \\ \mathbf{0}_{1 \times \gamma} & \mathbf{w} \mathbf{D} \mathbf{t}_i^\top \end{bmatrix}_s \in \mathbb{G}_s^{(k+\gamma+2) \times (\gamma+1)};$$

- Sample $\mathbf{W}_{1,i} \leftarrow \mathbb{Z}_p^{(2k+1) \times (\gamma+1)(k+1)}$, $\mathbf{W}_{2,i} \leftarrow \mathbb{Z}_p^{(2k+1) \times (k+1)(k+1)}$, $\mathbf{W}_{3,i} \leftarrow \mathbb{Z}_p^{(2k+1) \times (k+1)}$, $\mathbf{R}_i \leftarrow \mathbb{Z}_p^{(2k+2) \times (2k+1)}$, $\mathbf{r}_i \leftarrow \mathbb{Z}_p^{1 \times k}$;
- Execute $(\text{crs}_i, \text{td}_i) \leftarrow \text{LGen}(1^\lambda, \mathbb{G}_1, [\mathbf{A}_i]_1)$, where

$$\mathbf{A}_i = \begin{pmatrix} \mathbf{A} \\ \mathbf{R}_i \end{pmatrix} \in \mathbb{Z}_p^{(3k+2) \times (2k+1)};$$

- 4) Output the common reference string

$$\text{crs} = \begin{pmatrix} [\mathbf{A}, \mathbf{AV}_1, \mathbf{AV}_2, \mathbf{Av}^\top]_1, \\ \{\text{crs}_i, [\mathbf{R}_i, \mathbf{AW}_{1,i}, \mathbf{AW}_{2,i}, \mathbf{AW}_{3,i}]_1\}_{i \in [L]}, \\ \{[\mathbf{Dt}_i^\top, \mathbf{wDt}_i^\top, \mathbf{Br}_i^\top, \mathbf{W}_{1,i}(\mathbf{I}_{\gamma+1} \otimes \mathbf{Br}_i^\top) + \mathbf{V}_1]_2\}_{i \in [L]}, \\ \{[\mathbf{W}_{2,i}(\mathbf{Dt}_i^\top \otimes \mathbf{Br}_i^\top) + \mathbf{W}_{3,i}(\mathbf{wDt}_i^\top \otimes \mathbf{Br}_i^\top) + \mathbf{V}_2 \mathbf{Dt}_i^\top + \mathbf{v}^\top \mathbf{wDt}_i^\top]_2\}_{i \in [L]}, \\ \{[\mathbf{W}_{1,i}(\mathbf{I}_{\gamma+1} \otimes \mathbf{Br}_i^\top), \mathbf{W}_{2,i}(\mathbf{Dt}_i^\top \otimes \mathbf{Br}_i^\top) + \mathbf{W}_{3,i}(\mathbf{wDt}_i^\top \otimes \mathbf{Br}_i^\top)]_2\}_{i \in [L], j \in [L] \setminus \{i\}} \end{pmatrix}$$

KeyGen(crs, i) $\rightarrow$ (pk_i, sk_i): On input crs, each client i performs the following to generate a key pair (pk_i, sk_i):

- 1) Sample $\mathbf{U}_i \leftarrow \mathbb{Z}_p^{(2k+1) \times (k+1)}$ and set

$$\mathbf{F}_i = \begin{pmatrix} \mathbf{T}_i \\ \mathbf{Q}_i \end{pmatrix} = \begin{pmatrix} \mathbf{A} \\ \mathbf{R}_i \end{pmatrix} \mathbf{U}_i = \mathbf{A}_i \mathbf{U}_i \in \mathbb{Z}_p^{(3k+2) \times (k+1)};$$

- 2) Run $\pi_i \leftarrow \text{LPriv}(\text{crs}_i, [\mathbf{F}_i]_1, \mathbf{U}_i)$;
Parse $\{[\mathbf{Br}_j^\top]_2\}_{j \in [L]}$ from crs, output $\text{sk}_i = \mathbf{U}_i$ and

$$\text{pk}_i = \begin{pmatrix} \mathbf{T}_i = [\mathbf{AU}_i]_1, & \mathbf{Q}_i = [\mathbf{R}_i \mathbf{U}_i]_1, \\ \mathbf{h}_{i,j} = \{[\mathbf{U}_i \mathbf{Br}_j^\top]_2\}_{j \in [L] \setminus \{i\}}, & \pi_i \end{pmatrix}.$$

IsValid(crs, i, pk_i) $\rightarrow$ 0/1: To verify whether public key pk_i of any client $i \in [L]$ is valid, KC or others runs the following:

- 1) Parse $\text{pk}_i = ([\mathbf{T}_i, \mathbf{Q}_i]_1, \{[\mathbf{h}_{i,j}]_2\}_{j \in [L] \setminus \{i\}}, \pi_i)$;
- 2) Set $\mathbf{F}_i = \begin{pmatrix} \mathbf{T}_i \\ \mathbf{Q}_i \end{pmatrix}$ and check $\text{LVer}(\text{crs}_i, [\mathbf{F}_i]_1, \pi_i) \stackrel{?}{=} 1$;
- 3) For each $j \in [L] \setminus \{i\}$, check $e([\mathbf{A}]_1, [\mathbf{h}_{i,j}]_2) \stackrel{?}{=} e([\mathbf{T}_i]_1, [\mathbf{Br}_j]_2)$;
- 4) If all these checks pass, output 1; otherwise, output 0.

Aggr(crs, $\mathcal{I}, \{\text{pk}_i, \mathbf{y}_i\}_{i \in [L]}) \rightarrow (\text{mpk}, \{\text{hsk}_{\mathbf{y}_j}\}_{j \in [L]})$: KC provides registration services for the L clients as follows:

- 1) Parse $\text{pk}_i = ([\mathbf{T}_i, \mathbf{Q}_i]_1, \{[\mathbf{h}_{i,j}]_2\}_{j \in [L] \setminus \{i\}}, \pi_i)$ and $\mathbf{y}_i = (y_{i,1}, \dots, y_{i,r_i}) \in \mathbb{Z}_p^{r_i}$, where $i \in [L]$, $r_i = \text{poly}(\lambda)$ and $\mathcal{I} = \max_{i \in [L]} r_i$;
- 2) Sample $\mathbf{q}_i \leftarrow \mathbb{Z}_p^\gamma$ and add $(i, \mathbf{q}_i)$ to the (initially empty) public directory pd;
- 3) For $u \in [\mathcal{I}]$, set $\mathbf{y}_{i,u} = (y_{i,u})$ and have

$$\bar{\mathbf{y}}_{i,u} = \begin{cases} (\mathbf{y}_{i,u} \otimes \mathbf{q}_i, 1) & \text{if } u \leq r_i, \\ 0^{\gamma+1} & \text{if } u > r_i. \end{cases}$$

- 4) Compute $\mathbf{P}_0 = [\mathbf{A}]_1$, $\mathbf{P}_1 = [\mathbf{AV}_1]_1$,

$$\mathbf{P}_2 = [\mathbf{AV}_2]_1, \quad \mathbf{P}_3 = [\mathbf{Av}^\top]_1,$$

$$\mathbf{P}_{4,u} = \left[\sum_{i \in [L]} (\mathbf{T}_i + \mathbf{A}(\mathbf{W}_{1,i}(\bar{\mathbf{y}}_{i,u}^\top \otimes \mathbf{I}_{k+1}) + \mathbf{W}_{2,i}(\mathbf{Dt}_i \otimes \mathbf{I}_{k+1}) + \mathbf{W}_{3,i}(\mathbf{wDt}_i \otimes \mathbf{I}_{k+1}))) \right]_1;$$

- 3) For each $j \in [L]$, compute $\mathbf{k}_{j,0}^\top = [\mathbf{Br}_j^\top]_2$,

$$\mathbf{k}_{j,1,u}^\top = \left[\sum_{i \in [L] \setminus \{j\}} (\mathbf{h}_{i,j} + \mathbf{W}_{1,i}(\bar{\mathbf{y}}_{i,u}^\top \otimes \mathbf{Br}_j^\top) + \mathbf{W}_{2,i}(\mathbf{Dt}_i^\top \otimes \mathbf{Br}_j^\top) + \mathbf{W}_{3,i}(\mathbf{wDt}_i^\top \otimes \mathbf{Br}_j^\top)) \right]_2,$$

$$\mathbf{k}_{j,2,u}^\top = [\mathbf{W}_{1,j}(\bar{\mathbf{y}}_{j,u}^\top \otimes \mathbf{Br}_j^\top) + \mathbf{W}_{2,j}(\mathbf{Dt}_j^\top \otimes \mathbf{Br}_j^\top) + \mathbf{W}_{3,j}(\mathbf{wDt}_j^\top \otimes \mathbf{Br}_j^\top) + \mathbf{V}_1 \bar{\mathbf{y}}_{j,u}^\top + \mathbf{V}_2 \mathbf{Dt}_j^\top + \mathbf{v}^\top \mathbf{wDt}_j^\top]_2,$$

$$\mathbf{k}_{j,3}^\top = [\mathbf{Dt}_j^\top]_2, \quad \mathbf{k}_{j,4}^\top = [\mathbf{wDt}_j^\top]_2;$$

- 6) Output the master public key $\text{mpk} = (\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2, \mathbf{P}_3, \{\mathbf{P}_{4,u}\}_{u \in [\mathcal{I}]}, \text{pd})$ and helper decryption keys $\{\text{hsk}_{\mathbf{y}_j} = (\mathbf{k}_{j,0}^\top, \{\mathbf{k}_{j,1,u}^\top\}_{u \in [r_j]}, \{\mathbf{k}_{j,2,u}^\top\}_{u \in [r_j]}, \mathbf{k}_{j,3}^\top, \mathbf{k}_{j,4}^\top)\}_{j \in [L]}$.
 $\text{Enc}(\text{mpk}, \mathbf{x}, \mathcal{R}) \rightarrow \text{ct}_{\mathbf{x}}$: DO encrypts its message vector $\mathbf{x}$ for any clients outside $\mathcal{R}$ using mpk as follows:

- 1) Parse $\mathbf{x} = (x_1, \dots, x_m) \in \mathbb{Z}_p^m$ where $m = \text{poly}(\lambda)$, and set $\mathbf{x}_t = (x_t)$ for $t \in [m]$;
- 2) For all $j \in \mathcal{R}$, compute $\mathbf{e}_{\mathcal{R}} \in \mathbb{Z}_p^\gamma \setminus \{0\}$ such that $\langle \mathbf{q}_j, \mathbf{e}_{\mathcal{R}} \rangle = 0$;
- 3) Sample $\mathbf{s}_1, \dots, \mathbf{s}_m \leftarrow \mathbb{Z}_p^{1 \times k}$ and $\rho_1, \dots, \rho_m \leftarrow \mathbb{Z}_p$ such that $\sum_{t \in [m]} \rho_t = 0$;
- 4) For $t \in [m]$, set $\bar{\mathbf{x}}_t = (\mathbf{x}_t \otimes \mathbf{e}_{\mathcal{R}}, \rho_t)$ and compute

$$\mathbf{c}_{0,t} = [\mathbf{s}_t \mathbf{P}_0]_1 = [\mathbf{s}_t \mathbf{A}]_1,$$

$$\mathbf{c}_{1,t} = [\mathbf{s}_t \mathbf{P}_1 + \bar{\mathbf{x}}_t]_1 = [\mathbf{s}_t \mathbf{AV}_1 + \bar{\mathbf{x}}_t]_1,$$

$$\mathbf{c}_{2,t} = [\mathbf{s}_t \mathbf{P}_2]_1 = [\mathbf{s}_t \mathbf{AV}_2]_1,$$

$$\mathbf{c}_{3,t} = [\mathbf{s}_t \mathbf{P}_3]_1 = [\mathbf{s}_t \mathbf{Av}^\top]_1,$$

$$\mathbf{c}_{4,t} = [\mathbf{s}_t \mathbf{P}_{4,t}]_1$$

$$= \left[\sum_{i \in [L]} \mathbf{s}_t (\mathbf{T}_i + \mathbf{A}(\mathbf{W}_{1,i} (\bar{\mathbf{y}}_{i,t}^\top \otimes \mathbf{I}_{k+1}) + \mathbf{W}_{2,i} (\mathbf{D}\mathbf{t}_i^\top \otimes \mathbf{I}_{k+1}) + \mathbf{W}_{3,i} (\mathbf{w}\mathbf{D}\mathbf{t}_i^\top \otimes \mathbf{I}_{k+1}))) \right]_1;$$

5) Output $\mathbf{ct}_x = (\{\mathbf{c}_{0,t}, \mathbf{c}_{1,t}, \mathbf{c}_{2,t}, \mathbf{c}_{3,t}, \mathbf{c}_{4,t}\}_{t \in [m]}, \mathcal{R})$ and upload it to the WS.

Trace^D(mpk, $\mathcal{R}$, $\mu(\cdot)$, $\mathbf{x}_0^*$, $\mathbf{x}_1^*$) $\rightarrow \mathcal{L} \subseteq [L]$: To trace the traitor(s) in the system, CTA interacts with a μ -useful decoder D by proceeding the following:

- 1) For $i \in [L+1]$, set $\mathcal{S}_i = \{i, i+1, \dots, L+1\} \setminus \mathcal{R}$;
- 2) For each $j \in \mathcal{R}$, sample $\mathbf{e}_R \in \mathbb{Z}_p^\gamma \setminus \{\mathbf{0}\}$ such that $\langle \mathbf{q}_j, \mathbf{e}_R \rangle = \mathbf{0}$;
- 3) For $i = 1, \dots, L+1$, run:
 - If $i = 1$, let $\mathbf{e}_{\mathcal{S}_i} = \mathbf{0}$; If $i = L+1$, set $\mathbf{e}_{\mathcal{S}_i} = \mathbf{e}_R$; Otherwise, sample $\mathbf{e}_{\mathcal{S}_i} \in \mathbb{Z}_p^\gamma$ such that:
 - 1) $\langle \mathbf{q}_j, \mathbf{e}_{\mathcal{S}_i} \rangle = \mathbf{0}$ for every $j \in \mathcal{S}_i \cup \mathcal{R}$;
 - 2) $\langle \mathbf{q}_j, \mathbf{e}_{\mathcal{S}_i} \rangle = \langle \mathbf{q}_j, \mathbf{e}_R \rangle$ for every $j \in \mathcal{S}_i \setminus \mathcal{S}_i$;
 - Parse $\mathbf{x}_0^* = (x_{0,1}^*, \dots, x_{0,m}^*) \in \mathbb{Z}_p^m$, $\mathbf{x}_1^* = (x_{1,1}^*, \dots, x_{1,m}^*) \in \mathbb{Z}_p^m$, where $m = \text{poly}(\lambda)$;
 - For $t \in [m]$, set $\mathbf{x}_{0,t}^* = (x_{0,t}^*)$ and $\mathbf{x}_{1,t}^* = (x_{1,t}^*)$;
 - Let $\text{count}_i \leftarrow 0$. For $j = 1, \dots, \mathcal{L} = \lambda L^2 / \mu(\lambda)$, do:
 - 1) For $t \in [m]$, compute $\tilde{\mathbf{x}}_{i,t}^* = (\mathbf{x}_{1,t}^* - \mathbf{x}_{0,t}^*) \otimes \mathbf{e}_{\mathcal{S}_i} + \mathbf{x}_{0,t}^* \otimes \mathbf{e}_R \in \mathbb{Z}_p^{1 \times \gamma}$, sample $\rho_1^*, \dots, \rho_m^* \leftarrow \mathbb{Z}_p$ such that $\sum_{t \in [m]} \rho_t^* = 0$, set $\bar{\mathbf{x}}_{i,t}^* = (\tilde{\mathbf{x}}_{i,t}^*, \rho_t^*)$ and run $\mathbf{ct}_j^{S_i} = \text{Enc}(\text{mpk}, (\bar{\mathbf{x}}_{i,t}^*)_{t \in [m]}, \mathcal{R})$;
 - 2) Feed the decoder D with $\mathbf{ct}_j^{S_i}$ and obtain a binary value b_j^i . If $b_j^i = 0$, then $\text{count}_i \leftarrow \text{count}_i + 1$;
 - Output $\tilde{P}_i = \text{count}_i / L$.
- 4) Let $\mathcal{L}$ be the set of all $i \in [L]$ for which $|\tilde{P}_i - \tilde{P}_{i+1}| \geq \xi(\lambda)/L$ (note that if there is no such i , then $\mathcal{L} = \emptyset$), and output the set $\mathcal{L}$ as malicious user(s).

Dec(sk_{i^*} , $\text{hsk}_{\mathbf{y}_{i^*}}$, $\mathbf{ct}_x$) $\rightarrow \mathbf{xy}_{i^*} \cup \{\perp\}$: On input sk_{i^*} , $\text{hsk}_{\mathbf{y}_{i^*}}$ and $\mathbf{ct}_x$ from WS, the client i^* as DU decrypts $\mathbf{ct}_x$ as follows:

- 1) Parse $\mathbf{ct}_x = (\{\mathbf{c}_{0,t}, \mathbf{c}_{1,t}, \mathbf{c}_{2,t}, \mathbf{c}_{3,t}, \mathbf{c}_{4,t}\}_{t \in [m]}, \mathcal{R})$;
- 2) If $i^* \in \mathcal{R}$, then abort; Otherwise, for each $j \notin \mathcal{R}$, compute $\mathbf{e}_R \in \mathbb{Z}_p^\gamma \setminus \{\mathbf{0}\}$ such that $\langle \mathbf{e}_R, \mathbf{q}_j \rangle \neq \mathbf{0}$;
- 3) Parse $\text{sk}_{i^*} = \mathbf{U}_{i^*}$, $\text{hsk}_{\mathbf{y}_{i^*}} = (\mathbf{k}_{i^*,0}^\top, \{\mathbf{k}_{i^*,1,u}^\top\}_{u \in [r_{i^*}]}, \{\mathbf{k}_{i^*,2,u}^\top\}_{u \in [r_{i^*}]}, \mathbf{k}_{i^*,3}^\top, \mathbf{k}_{i^*,4}^\top)$, where $\mathbf{y}_{i^*} = (y_{i^*,1}, \dots, y_{i^*,r_{i^*}}) \in \mathbb{Z}_p^{r_{i^*}}$ and $\mathbf{x} = (x_1, \dots, x_m) \in \mathbb{Z}_p^m$;
- 4) If $[r_{i^*}] \not\subseteq [m]$, outputs $\perp$. Otherwise, for $u = t \in [m]$,

$$\begin{aligned} [z_{1,t}]_T &= e([\mathbf{c}_{4,t}]_1, [\mathbf{k}_{i^*,0}^\top]_2), & [z_{2,t}]_T &= e([\mathbf{c}_{0,t}]_1, [\mathbf{k}_{i^*,1,t}^\top]_2), \\ [z_{3,t}]_T &= e([\mathbf{c}_{0,t} \mathbf{U}_{i^*}]_1, [\mathbf{k}_{i^*,0}^\top]_2), & [z_{4,t}]_T &= e([\mathbf{c}_{0,t}]_1, [\mathbf{k}_{i^*,2,t}^\top]_2), \\ [z_{5,t}]_T &= e([\mathbf{c}_{1,t}]_1, [\bar{\mathbf{y}}_{i^*,t}^\top]_2), & [z_{6,t}]_T &= e([\mathbf{c}_{2,t}]_1, [\mathbf{k}_{i^*,3}^\top]_2), \\ [z_{7,t}]_T &= e([\mathbf{c}_{3,t}]_1, [\mathbf{k}_{i^*,4}^\top]_2); \end{aligned}$$

5) Compute

$$[z]_T = [(z_{1,t} - z_{2,t} - z_{3,t} - z_{4,t} + z_{5,t} + z_{6,t} + z_{7,t})]_T.$$

- 6) Recover and output $z = \langle \mathbf{x}, \mathbf{y}_{i^*} \rangle$ from $[z]_T = [\sum_{t \in [m]} z_t / \langle \mathbf{e}_R, \mathbf{q}_{i^*} \rangle]_T$ via brute-force DLOG¹ if $\|\mathbf{x}\|_\infty \leq \mathcal{X}_\lambda$, $\|\mathbf{y}_{i^*}\|_\infty \leq \mathcal{Y}_\lambda$ and $|\sum_{t \in [m]} x_t y_{i^*,t}| \leq m \mathcal{X}_\lambda \mathcal{Y}_\lambda$.

Completeness and Correctness: Due to the space limitations, we prove the completeness and correctness of our TRFE-UIP scheme in Appendix A (available online).

Theorem 1: Assuming that $\Pi_0 = (\text{LGen}, \text{LPrv}, \text{LVer}, \text{LSim})$ is a QA-NIZK with perfect completeness, perfect zero-knowledge and unbounded simulation soundness for linear space defined in **Definition 3**, our TRFE-UIP scheme achieves the Sel*-SIM security under MDDH assumption. More precisely, if there exists any PPT adversary $\mathcal{A}_1$ that breaks the Sel*-SIM security of our TRFE-UIP scheme, then we can construct PPT machines $\mathcal{B}_1, \mathcal{B}_2$ such that:

$$\begin{aligned} \text{Adv}_{\mathcal{A}_1}^{\text{TRFE-UIP}}(\lambda) &\leq L \cdot \text{Adv}_{\mathcal{B}_1}^{\text{USS}}(\lambda) + (2L + 2LQ + 1) \cdot \\ &\quad \text{Adv}_{\mathcal{B}_2}^{\text{MDDH}}(\lambda) + \text{negl}(\lambda) \end{aligned}$$

where λ is the security parameter, L is the number of slots, Q is the maximum number of queries on a slot made by $\mathcal{A}_1$, and $\text{Time}(\mathcal{B}_1), \text{Time}(\mathcal{B}_2) \approx \text{Time}(\mathcal{A}_1)$.

Theorem 2: Assuming that our TRFE-UIP scheme satisfies Sel*-SIM security under MDDH assumption, then our scheme also satisfies public black-box traceability as defined in **Definition 9**. More precisely, if there exists any PPT adversary $\mathcal{A}_2$ that breaks the public black-box traceability of our scheme, then we can construct a PPT algorithm $\mathcal{B}$ that breaks the Sel*-SIM security of our scheme with a non-negligible probability.

Remark 3: Considering the paper space requirement, we give the specific proofs of Theorems 1 and 2 in Appendix A (available online).

V. PERFORMANCE EVALUATION

We evaluate the performance of our TRFE-UIP scheme by comparing it with six existing IPFE schemes. For convenience, we denote the unbounded IPFE (UIPFE) schemes proposed by Tomida et al. [16], Yang et al. [20], Zhang et al. [27] as [TT18], [YG24] and [ZL24], respectively. Similarly, the traceable IPFE (TIPFE) schemes introduced by Do et al. [21], Qiu et al. [24], and Branco et al. [29] are denoted as [DP20], [QH24], and [BL24], respectively. We simulate all comparative schemes on a laptop equipped with an Intel Core i5-4210 U CPU @1.7 GHz, 2 GB RAM, and memory and Ubuntu 64 OS. The codes are written using C language based on MIRACL [34]. We employ a Type F elliptic curve with an embedding degree $k = 12$ to construct the asymmetric bilinear mapping $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. This curve is defined by the equation $y^2 = x^3 + b$ over the finite field $\mathbb{F}_p$.

¹In the Dec algorithm, $z = \langle \mathbf{x}, \mathbf{y}_{i^*} \rangle$ satisfies $|z| \leq N = m \mathcal{X}_\lambda \mathcal{Y}_\lambda$. Thus, z can be recovered via bounded exhaustive search over the interval $[-N, N]$. The decryption approach follows the prior FE schemes [8], [10], [12], [25], [33], where the boundedness of the message space guarantees the feasibility of the otherwise intractable discrete logarithm computation. In practice, the discrete logarithm can be solved by the Baby-step Giant-step (BSGS) method, which reduces the time complexity from $O(N)$ to $O(\sqrt{N})$, at the cost of $O(\sqrt{N})$ storage space. For a concrete instantiation with $\mathcal{X}_\lambda = \mathcal{Y}_\lambda = 2^{10}$ and $m = 50$, we have $N \approx 5.24 \times 10^7$ and $\sqrt{N} \approx 7.24 \times 10^3$. Thus, BSGS requires roughly 7.2×10^3 group operations and 7.2×10^3 storage entries (tens of KB), which is negligible on modern hardware.

TABLE II
NOTATIONS IN PERFORMANCE EVALUATION

Notations	Descriptions
NKE	No key escrow problem
UIP	Unbounded inner product
TMU	Tracing malicious users
PP	Privacy-preservation
SAD-SIM	Semi-adaptive simulation-based security
SAD-IND	Semi-adaptive indistinguishability
AD Q -SIM	Adaptive simulation-based security under prior-bound Q
Sel-IND	Selective indistinguishability
Sel*-SIM	Very selective simulation-based security
$ \mathbb{G}_1 $	Size of an element in $\mathbb{G}_1$
$ \mathbb{G}_2 $	Size of an element in $\mathbb{G}_2$
$ \mathbb{G}_T $	Size of an element in $\mathbb{G}_T$
$ \mathbb{Z}_p $	Size of an element in $\mathbb{Z}_p$
T_{e1}	Execution time of an exponential over $\mathbb{G}_1$
T_{e2}	Execution time of an exponential over $\mathbb{G}_2$
T_{eT}	Execution time of an exponential over $\mathbb{G}_T$
T_p	Execution time of a bilinear pairing

*The collusion bound Q , implied by the security parameter 1^λ , is an integer much less than the total number of system users L , which means not many users collude with adversary.

TABLE III
FUNCTIONALITY AND SECURITY PROPERTIES COMPARISON

Scheme	Functionality				Security Model		
	NKE	UIP	TMU	PP	Security	Traceability	Assumption
[TT18]	×	✓	-	×	SAD-IND	-	SXDH
[YG24]	×	✓	-	×	SAD-SIM	-	SXDH
[ZL24]	✓	✓	-	✓	AD Q -SIM	-	k-Lin
[DP20]	×	-	✓	×	Sel-IND	Black box	BDDH
[QH24]	×	-	✓	✓	Sel-IND	White box	DDH
[BL24]	✓	-	✓	×	Sel-IND	Black box	DDH
Ours	✓	✓	✓	✓	Sel*-SIM	Black box	MDDH

According to the parameter generation algorithm proposed by Barreto and Naehrig [35], the group order p is a 256-bit prime number to achieve a security level of $\lambda = 128$ bits. Table II describes the notations used in the comparisons.

A. Functionality and Security Properties

Table III summarizes the functional properties and security of our scheme and the IPFE schemes [16], [20], [21], [24], [27], [29]. From the Table III, we can see that the IPFE schemes in [TT18], [YG24] and [ZL24] only support unbounded inner product, while those in [DP20], [QH24] and [BL24] solely achieve traitor tracing. Our TRFE-UIP scheme is the only one that provides both unbounded inner product computation and traitor tracing functionalities. Like those of [ZL24] and [BL24], Our scheme is an RFE scheme based on registration-based encryption, which can effectively avoid the key escrow problem in FE systems. In addition, the traitor tracing system needs to consider that user privacy and anonymity may be compromised by traceability. [QH24] employs a commitment scheme and zero-knowledge proof to hide the user identity, preventing even the key generation center from learning specific identity information. In comparison, our scheme with [ZL24] leverages slot indices to anonymize real user identities, where slot indices are uncorrelated with user identities. This design ensures that no adversary cannot derive real user identity from the acquired

slot index and secret key information, thereby preserving user privacy effectively.

Regarding security guarantees, [TT18] and [YG24] achieve SAD-IND and SAD-SIM security under the SXDH assumption, respectively. [ZL24] meets the AD Q -SIM security under the k -Lin assumption, but its reliance on garbled circuits makes it inapplicable for practical deployment. With the support of public black-box traceability, [DP20] and [BL24] satisfy the Sel-IND security under BDDH and DDH assumptions, respectively. [QH24] realizes white-box traceability, but it only meets the Sel-IND security under the DDH assumption.

We should note that the SIM security is inherently stronger than IND security, because it requires the existence of a simulator that prevents any adversary from distinguishing between a real execution and a simulated execution, unlike IND security which only requires that any adversary cannot distinguish between two challenge messages. Moreover, SIM security here involves three types: Sel*-SIM, SAD-SIM, and AD Q -SIM. Their essential differences lie in the adversary's capabilities and the challenge query phase. In Sel*-SIM security, an adversary must pre-determine the challenge target (e.g., x^*) before the system initialization, and a simulator is required to generate a simulated view indistinguishable from the real environment with almost no prior information. In contrast, SAD-SIM allows an adversary to adaptively select some target information after the system setup but before the challenge phase, offering greater flexibility than the Sel*-SIM security model. Furthermore, AD Q -SIM security allows an adversary to make adaptive selections, only requiring that the number of challenges be limited by the prior bound Q . Thus, the AD Q -SIM security is the strongest, but its security proof usually requires additional complex tools (such as garbled circuit).

In conclusion, our TRFE-UIP scheme achieves Sel*-SIM security under the MDDH assumption when compared to the existing IND-secure schemes. Although the security of our scheme is weaker than [YG24], it further supports public black-box traceability and proves that the process avoids the reliance on obfuscation circuits required by [ZL24].

B. Communication Cost Analysis

Given that the security of all compared schemes relies on different hardness assumptions, we conduct a fair performance comparison by aligning all schemes at an identical security level. As is known in **Definition 1**, the above assumptions are inter-convertible under certain conditions. The security of our TRFE-UIP scheme is based on the MDDH assumption, a generalized variant of the k -Lin assumption. Thus, we instantiate our TRFE-UIP scheme and [ZL24] with two configurations by adjusting the parameter k : Ours-1 and [ZL24]-1 for $k = 1$ while Ours-2 and [ZL24]-2 for $k = 2$. Since the compared UIPFE schemes do not rely on the BDDH assumption, [ZL24]-2 is omitted in the following comparisons. For fairness, we perform the subsequent experimental analysis with strict security-level matching: Ours-1 is compared with [TT18], [YG24], [ZL24] (namely [ZL24]-1), [QH24] and [BL24] at the same security level, while Ours-2 is compared with [DP20].

TABLE IV
COMMUNICATION COSTS OF UNBOUNDED IPFE SCHEMES (BYTES OR B)

Scheme	Public parameter	Key for Encryption	Key for Decryption	Ciphertext	Total
[TT18]	$4L \mathbb{G}_1 $	$4L \mathbb{G}_1 $	$7n \mathbb{G}_2 $	$7n \mathbb{G}_1 $	$(7n + 8L) \mathbb{G}_1 + 7n \mathbb{G}_2 $
[YG24]	$8L \mathbb{G}_1 + 2L \mathbb{G}_T $	$8L \mathbb{G}_1 + 2L \mathbb{G}_T $	$4n \mathbb{G}_2 + 2 \mathbb{Z}_q $	$5n \mathbb{G}_1 + 2 \mathbb{G}_T $	$(5n + 16L) \mathbb{G}_1 + 4n \mathbb{G}_2 + (4L + 2) \mathbb{G}_T + 2 \mathbb{Z}_q $
[ZL24]	$4nL \mathbb{G}_1 + 4n(L^3 + 2L^2) \mathbb{G}_2 + 2nL \mathbb{G}_T $	$8nL \mathbb{G}_1 + 2nL \mathbb{G}_T $	$10nL \mathbb{G}_2 + 8nL \mathbb{Z}_p $	$12nL \mathbb{G}_1 $	$24nL \mathbb{G}_1 + (4L^2 + 8L + 10)nL \mathbb{G}_2 + 4nL \mathbb{G}_T + (15L + 10n + 9) \mathbb{G}_1 + (24L + 4n + 6) \mathbb{G}_2 + 6 \mathbb{Z}_q $
Ours-1	$(15L + 6) \mathbb{G}_1 + 24L \mathbb{G}_2 $	$(2n + 4) \mathbb{G}_1 $	$(4n + 6) \mathbb{G}_2 + 6 \mathbb{Z}_p $	$8n \mathbb{G}_1 $	

TABLE V
COMMUNICATION COSTS OF TRACEABLE IPFE SCHEMES (BYTES OR B)

Scheme	Public parameter	Key for Encryption	Key for Decryption	Ciphertext	Total
[DP20]	$(n + 1) \mathbb{G}_1 + \mathbb{G}_2 + (n + 1) \mathbb{G}_T + nL \mathbb{Z}_p $	$(n + 1) \mathbb{G}_1 + \mathbb{G}_2 + (n + 1)L \mathbb{G}_T + n \mathbb{Z}_p $	$n \mathbb{G}_2 $	$n \mathbb{G}_1 + n \mathbb{G}_T $	$(3n + 2) \mathbb{G}_1 + (n + 2) \mathbb{G}_2 + (nL + 2n + L + 1) \mathbb{G}_T + (L + 1)n \mathbb{Z}_p $
[QH24]	$(n + 5) \mathbb{G}_1 + 2 \mathbb{G}_2 + (n + 2) \mathbb{Z}_p $	$(n + 5) \mathbb{G}_1 + 2 \mathbb{G}_2 $	$3 \mathbb{G}_1 + \mathbb{G}_2 + 2 \mathbb{Z}_p $	$(n + 3) \mathbb{G}_1 + \mathbb{G}_2 + \mathbb{Z}_p $	$(3n + 16) \mathbb{G}_1 + 6 \mathbb{G}_2 + (n + 5) \mathbb{Z}_p $
[BL24]	$L \mathbb{G}_1 + 2L^2 \mathbb{G}_2 + L \mathbb{Z}_p $	$2n \mathbb{G}_1 $	$(n + 2) \mathbb{G}_2 + n \mathbb{Z}_p $	$(n + 2) \mathbb{G}_1 $	$(3n + L + 2) \mathbb{G}_1 + (2L^2 + n + 2) \mathbb{G}_2 + (L + n) \mathbb{Z}_p $
Ours-1	$(2L^2 + 14L + 5) \mathbb{G}_1 + (3L^2 + 11L) \mathbb{G}_2 $	$(2n + 4) \mathbb{G}_1 + (L + 2) \mathbb{Z}_p $	$(4n + 6) \mathbb{G}_2 + 6 \mathbb{Z}_p $	$8n \mathbb{G}_1 $	$(2L^2 + 14L + 10n + 9) \mathbb{G}_1 + (3L^2 + 11L + 4n + 6) \mathbb{G}_2 + (L + 8) \mathbb{Z}_p $
Ours-2	$(5L^2 + 40L + 12) \mathbb{G}_1 + (5L^2 + 47L) \mathbb{G}_2 $	$(3n + 12) \mathbb{G}_1 + (2L + 4) \mathbb{Z}_p $	$(6n + 9) \mathbb{G}_2 + 15 \mathbb{Z}_p $	$10n \mathbb{G}_1 $	$(5L^2 + 40L + 13n + 24) \mathbb{G}_1 + (5L^2 + 47L + 6n + 9) \mathbb{G}_2 + (2L + 19) \mathbb{Z}_p $

Since [ZL24], [BL24], and our scheme are based on RFE, they support L registered clients sharing a single mpk for encryption. And mpk is generated by aggregating all registered clients' public keys. A registered client, as DU, requires both a private key sk and a helper decryption key hsk for decryption. Hence, we refer to sk and hsk together as the decryption key. Furthermore, [DP20], based on multi-client functional encryption (MCFE), can be transformed into an FE system for L users. In contrast, [TT18], [YG24] and [QH24] support a single-user environment, requiring L invocations of the Setup algorithm to generate public parameters (i.e., encryption keys) for L users, with the decryption key being the DU's private key. The communication costs of all schemes are measured in terms of group elements ($\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$) and finite field elements ($\mathbb{Z}_p$), where $|\mathbb{G}_1| = 32$ B, $|\mathbb{G}_2| = 64$ B, $|\mathbb{G}_T| = 384$ B, and $|\mathbb{Z}_p| = 20$ B. The communication costs of each scheme show varying degrees of correlation with the message vector dimension n and the number of client registration L .

Table IV evaluates the communication costs of the compared UIPFE schemes. We can clearly observe that Ours-1 scheme achieves an optimal trade-off between communication efficiency and functional completeness. The length of its public parameter is $(15L + 6)|\mathbb{G}_1| + 24L|\mathbb{G}_2|$, and the sizes of encryption key, decryption key and ciphertext are only linearly correlated with n . The total communication overhead is $(15L + 10n + 9)|\mathbb{G}_1| + (24L + 4n + 6)|\mathbb{G}_2| + 6|\mathbb{Z}_p|$. Through structural optimization, our scheme completely eliminates the high-order term dependence on L . Although the public parameter overhead of Ours-1 scheme is slightly higher than that of [TT18] and [YG24], this minor extra cost is a reasonable compromise for achieving richer functional support and superior scalability. In contrast, our scheme achieves a substantial

reduction in the core encryption key overhead. All overhead terms of Ours-1 scheme maintain a gentle linear growth trend with the increase of n and L , which enables our scheme to outperform all the existing counterparts in comprehensive performance for practical large-scale multi-user and high-dimensional vector scenarios.

In addition, our scheme is equipped with traitor functionality. For the existing TIPFE schemes, their communication overhead incorporates extra components induced by the traitor mechanism on top of the overhead of UIPFE schemes. As illustrated in Table V, we conduct the communication cost comparisons between our proposed scheme and the existing TIPFE schemes. At the same security level, Ours-2 scheme exhibits prominent advantages over [DP20]. Specifically, the sizes of the public parameter and encryption key in [DP20] both involve high-complexity nL coupling terms, with their dominant terms being $(n - 1)|\mathbb{G}_T|$ and $(n + 1)L|\mathbb{G}_T|$, respectively. This causes the communication overhead of [DP20] to increase linearly with the number of clients L , which severely restricts its practicability in large-scale user scenarios. By contrast, the public parameter size of Ours-2 scheme follows an $\mathcal{O}(L^2)$ structure. Although the encryption key size of Ours-2 scheme is linearly correlated with both n and L , its dominant term is only $(3n + 12)|\mathbb{G}_1|$ without redundant overheads in the group $\mathbb{G}_T$. Moreover, the sizes of the decryption key and ciphertext in Ours-2 scheme are only linearly related to n . Although the length of the public parameter in Ours-2 scheme is slightly higher than that in [DP20], this is a necessary trade-off for achieving the dual core functionalities of unbounded inner product evaluation and traitor tracing, in return for superior adaptability to large-scale application scenarios.

When comparing Ours-1 scheme with [QH24] and [BL24] at the same security level, the prominent advantage of balancing

TABLE VI
COMPUTATION COSTS OF UNBOUNDED IPFE SCHEMES (MS)

Scheme	Setup	KeyGen	Encrypt	Decrypt	Total
[TT18]	$4LT_{e_1}$	$7nLT_{e_2}$	$4nT_{e_1}$	nT_p	$4(L+n)T_{e_1} + 7nLT_{e_2} + nT_p$
[YG24]	$8LT_{e_1} + 2LT_{e_T}$	$4nLT_{e_2}$	$8nT_{e_1}$	nT_p	$8(L+n)T_{e_1} + 4nLT_{e_2} + 2LT_{e_T} + nT_p$
[ZL24]	$16nLT_{e_1} + (2nL^2 + 2(L-1))T_{e_2} + 2nT_{e_T}$	$16nT_{e_1} + (14L-2)nT_{e_2}$	$2n(1+L)T_{e_1} + 2nT_{e_T}$	$2nT_{e_T} + 10nT_p$	$(16L+18)nT_{e_1} + (2nL^2 + 8nL + 8L - 2n - 2)T_{e_2} + 4nT_{e_T} + 10nT_p$
Ours-1	$27LT_{e_1} + 25LT_{e_3}$	$(2nL+17)T_{e_1} + (6nL+8L-2)T_{e_2}$	$2L+8nT_{e_1}$	$7nT_p$	$(4nL+27L+8n+17)T_{e_1} + (6nL+33L-2)T_{e_2} + 7nT_p$

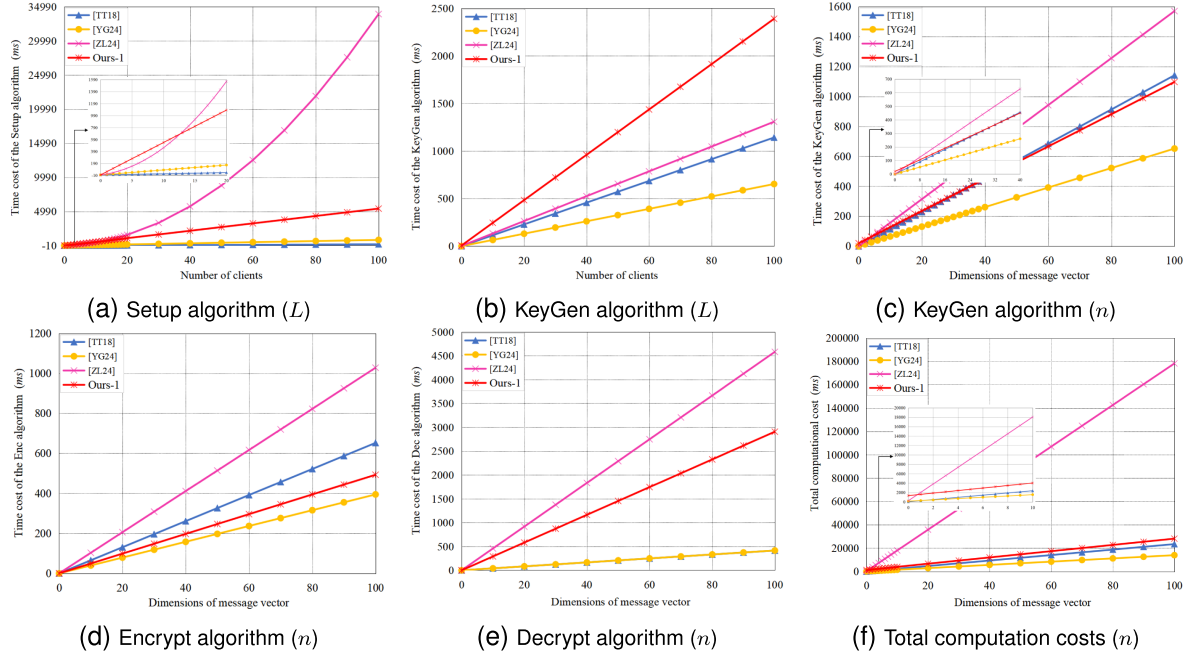


Fig. 3. Computation cost comparison of our scheme and three UIPFE schemes.

functional integrity and performance is highlighted. Specifically, [QH24] has a linear structure in public parameters with low decryption key overhead, yet it lacks the traitor functionality and thus suffers from an essential functional deficiency. The public parameter size of [BL24] is also in the order of $\mathcal{O}(L^2)$, and the coefficient of its $2L^2|\mathbb{G}_2|$ term is significantly lower than that of the $3L^2|\mathbb{G}_2|$ term in Ours-1 scheme. Nevertheless, the encryption key of Ours-1 scheme is $(2n+4)|\mathbb{G}_1| + (L+2)|\mathbb{Z}_p|$, which is slightly larger than the $2n|\mathbb{G}_1|$ of [BL24] but avoids the n -related redundant terms in the decryption key of [BL24]. Although the decryption key of Ours-1 scheme is marginally longer than that of [QH24], our scheme achieves a breakthrough in traitor tracing functionality. Furthermore, the ciphertext of Ours-1 scheme only consists of group elements in $\mathbb{G}_1$, with a concise structure and superior stability in high-dimensional scenarios with increasing n . In summary, our scheme achieves superior efficiency in core communication modules (i.e., encryption key and ciphertext), while filling the functional gaps of the existing schemes by supporting both unbounded inner product evaluation and traitor tracing with enhanced security guarantees.

C. Computation Cost Analysis

We primarily consider the computational overhead of the Setup, KeyGen, Encrypt, and Decrypt algorithms. For TIPFE schemes, the computational overhead of the Trace algorithm is additionally considered. For precise and comparable results, all schemes are tested under a unified parameter configuration ($\lambda = 256$, $L = 100$, $n = 100$, $\mu(\lambda) = 0.05$). To obtain more accurate experimental data, each test item is independently executed 100 times and the average value is adopted for subsequent analysis.

Table VI and Fig. 3 present the comparison of computational costs between Ours-1 scheme and the state-of-the-art UIPFE schemes. The computational cost of each algorithm is evaluated by the sum of the number of basic operations involved in the algorithm execution. As shown in Table VI, the computational time of the Setup algorithm in Ours-1 scheme is only linearly correlated with the number of clients L . The KeyGen algorithm consists of the key generation algorithms for L users and the aggregation algorithm, whose time consumption is positively correlated with both n and L , and the computational

TABLE VII
COMPUTATION COSTS OF TRACEABLE IPFE SCHEMES (MS)

Scheme	Setup	KeyGen	Encrypt	Decrypt	Trace	Total
[DP20]	$nT_{e_1} + nT_{e_T} + T_p$	T_{e_2}	$3nT_{e_T}$	$nT_{e_1} + nT_{e_T} + T_p$	$\frac{8\lambda nL^2(T_{e_1} + 2T_{e_T})}{\mu(\lambda)}$	$2nT_{e_1} + T_{e_2} + 5nT_{e_T} + 2T_p$
[QH24]	$(n+2)LT_{e_1}$	$(nL+11L)T_{e_1} + 9LT_p$	$(2n+3)T_{e_1}$	$(n+2)T_{e_1} + 5T_p$	$3T_{e_1} + 4T_p$	$(2nL+13L+3n+5)T_{e_1} + (9L+9)T_p$
[BL24]	$LT_{e_1} + L^2T_{e_2}$	$(L^2+n)T_{e_1} + (L^2+nL-n)T_{e_2} + LT_p$	$3nT_{e_1}$	$3T_p + 2T_{e_T}$	$\frac{8L^3(3T_{e_1}+3T_p+T_{e_T})}{\mu(\lambda)^2}$	$(L^2+L+4n)T_{e_1} + (2L^2+nL-n)T_{e_2} + 2T_{e_T} + LT_p$
Ours-1	$(2L+25)T_{e_1} + (3L+21)T_{e_2}$	$(2nL+17)T_{e_1} + (6nL+8L-2)T_{e_2}$	$(3nL+7n)T_{e_1}$	$7nT_p$	$\frac{4\lambda nL^4T_{e_1}}{\mu(\lambda)}$	$(5nL+31L+7n+42)T_{e_1} + (6nL+11L+19)T_{e_2} + 7nT_p$
Ours-2	$(6L+68)T_{e_1} + (5L+30)T_{e_2}$	$(3nL+2L+39)T_{e_1} + (6nL+12L-3)T_{e_2}$	$(4nL+10n)T_{e_1}$	$7nT_p$	$\frac{6\lambda nL^4T_{e_1}}{\mu(\lambda)}$	$(7nL+81+10n+107)T_{e_1} + (6nL+17L+27)T_{e_2} + 7nT_p$

overhead of generating a single ciphertext is $(2nL+8n)T_{e_1}$. In addition, the time consumption of the Decrypt algorithm in Ours-1 scheme is $7nT_p$. When comparing the computational costs with other UIPFE schemes, we first take into account the factors that lead to higher computational overheads of the algorithms in Ours-1 scheme, then fix the corresponding variables as constant values under the worst-case scenario of our scheme, and finally set the independent variables for fair comparison.

As illustrated in Fig. 3(a), with the number of clients fixed as $L=100$, the computational overhead of the Setup algorithm in Ours-1 scheme is approximately 5408.6 ms for varying message vector dimensions n , which outperforms the $33413.072n+322.938$ ms of [ZL24], yet is higher than 197.2 ms of [TT18] and 821.6 ms of [YG24]. In Fig. 3(b) and (c), with $L=100$, the computational time of executing the KeyGen algorithm once in Ours-1 scheme is $1077.2n+1309.919$ ms, which is larger than $652.4n$ ms of [YG24]. Specifically, Ours-1 scheme outperforms [ZL24] with $2296.626n$ ms when $n \geq 2$, and surpasses [TT18] with $1141.7n$ ms when $n \geq 21$. Moreover, it can be clearly observed from Fig. 3(d) and (e) that with $n=100$, the time consumed by Ours-1 scheme to encrypt a message that supports decryption for L clients is $98.6L+394.4$ ms. Although this overhead is higher than 394.4 ms of [YG24] and 197.2 ms of [TT18], Ours-1 scheme consistently outperforms [ZL24] with $2283.4L+462.6$ ms, while its growth rate remains well within an acceptable range.

Overall, under the parameter setting of $L=100$ and $n=100$, the total computational overhead of Ours-1 scheme in Fig. 3(f) is approximately 127600.719 ms (1.276×10^3 ss). This represents a reduction of 41.2% compared with 114979.8 ms of [TT18], a 38.3% reduction compared with 66871.4 ms of [YG24], and a remarkable 96.4% reduction compared with 3586259.738 ms of [ZL24]. Our scheme achieves a substantial improvement in computational efficiency while retaining the core functionality of unbounded inner product evaluation, and this dominant advantage is sufficient to offset the slight overhead differences in specific scenarios.

In addition, the computational overhead of a TIPFE scheme essentially consists of the basic overhead of the underlying IPFE scheme plus the additional time cost incurred by the traitor tracing mechanism. Table VII and Fig. 4 present the

computational cost comparisons between our proposed scheme and the existing TIPFE schemes. From Table VII, when $k=1$, after incorporating the parameters for traitor tracing, Ours-1 scheme requires $(2L+25)T_{e_1} + (3L+21)T_{e_2}$ for the Setup algorithm, the computational cost for encrypting an n -dimensional message becomes $(3nL+7n)T_{e_1}$, while the Trace algorithm incurs a cost of $\mathcal{O}(L^5)$. The computational costs of the KeyGen and Enc algorithms remain the same as those in the baseline setting. For $k=2$, the Setup algorithm of Ours-2 scheme takes $(6L+68)T_{e_1} + (5L+30)T_{e_2}$, while the computational overhead of the KeyGen algorithm is positively correlated with n and L . The encryption time is $(4nL+10n)T_{e_1}$, the decryption cost is consistent with that of Ours-1, and its computational overhead for tracing a traitor is slightly higher than that of Ours-1. Subsequently, a refined comparison is conducted between our proposed scheme and the TIPFE schemes under the same security level.

As illustrated in Fig. 4(a)–(b), when $L=100$, the computational cost of the Setup algorithm in Our-1 scheme is approximately 634.476 ms, which is only 3.89% of 16359.3 ms in [BL24]. For $n > 10$, Ours-1 scheme outperforms [QH24] significantly, which requires $49.3n+98.6$ ms. Ours-2 scheme incurs 1,194.754 ms for the Setup algorithm. Although it is slightly inferior to $2.629n+4.154$ ms of [DP20] in low-dimensional scenarios, it achieves superior performance in high-dimensional settings ($n > 452$), demonstrating excellent adaptability for high-dimensional data. In Fig. 4(c)–(d), with $L=100$, the computational time for a single execution of the KeyGen algorithm in Our-1 scheme is close to that of [QH24], and it consistently outperforms [ZL24]’s $2296.626n$ ms for $n > 9$. The KeyGen algorithm of Our-2 scheme requires $1126.5n+2070.134$ ms, which is higher than the 1.631 ms of [DP20]. However, this increased overhead is a necessary trade-off for realizing the unbounded vector feature.

Furthermore, as shown in Fig. 4(e), for $n=100$, the computational time for a single encryption in Our-1 scheme is less efficient compared to the $0.986n+1.479$ ms of [QH24] and the $1.479n$ ms of [BL24]. The encryption cost for Our-2 scheme is $202.13n$ ms, which is also higher than the $6.408n$ ms of [DP20]. In Fig. 4(f), Our-1 scheme demonstrates an advantage over $0.493n+21.756$ ms of [QH24] and $1.479n$ ms of [BL24] only when decrypting a ciphertext corresponding to a one-dimensional message. To decrypt a ciphertext, Our-2 scheme

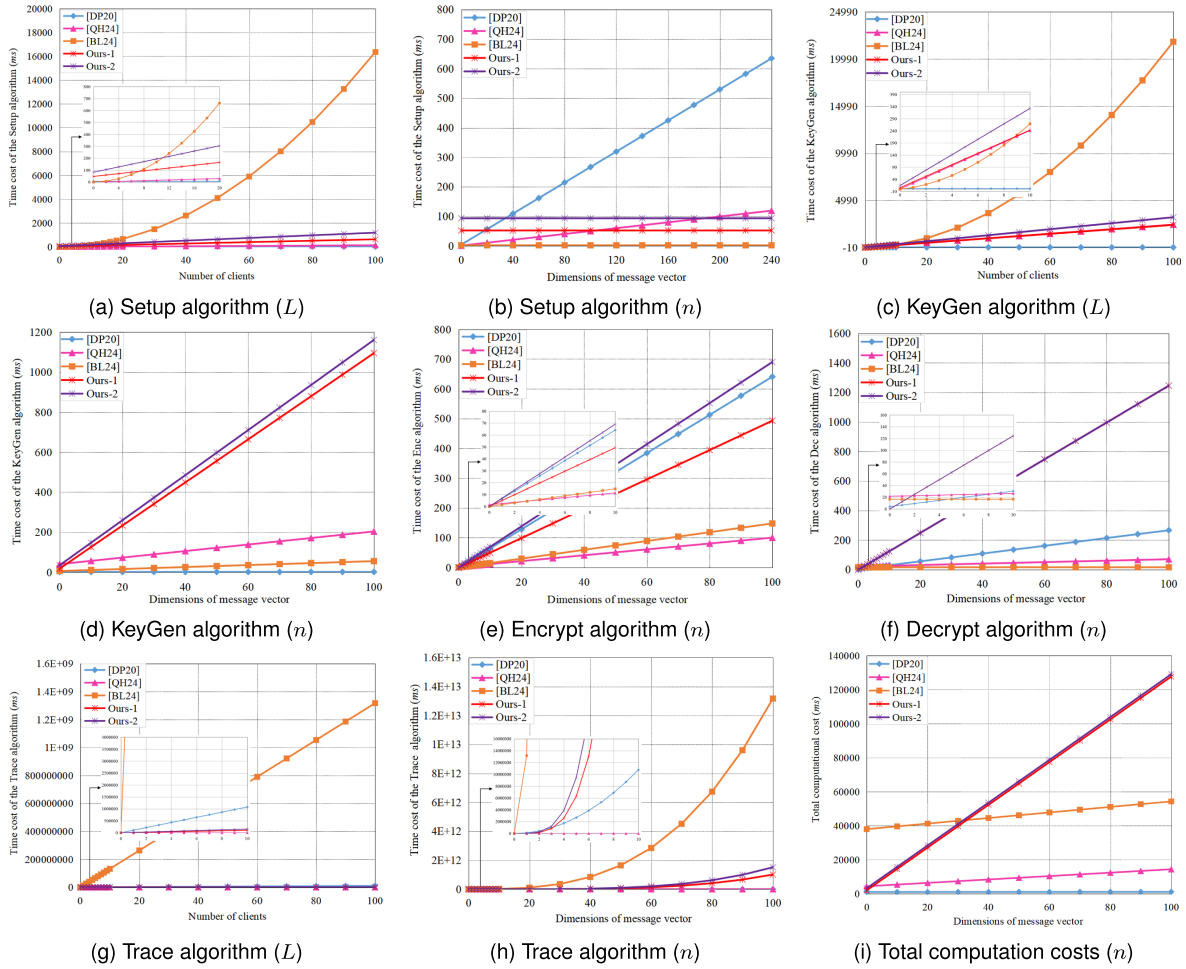


Fig. 4. Computation cost comparison of our scheme and three TIPFE schemes

takes about $202.13n$ ms, compared to $2.629n + 4.154$ ms for [DP20]. This discrepancy stems from the functional extension in Our-2 to support unbounded vectors without $i\mathcal{O}$.

Figure 4(g)–(h) reveal that for $L = 100$, Our-1 scheme requires about $1.015n \times 10^{11}$ ms for traitor tracing, which is better than $1.315n \times 10^{12}$ ms of [BL24] but inferior to the 18.095 ms of [QH24]. The tracing algorithm of the Our-2 scheme takes $1.522n \times 10^{11}$ ms, which is less efficient than the $1.941n \times 10^9$ ms of [DP20]. Although our scheme may not outperform others in terms of computational overhead for individual algorithms, the overall burden in Fig. 4(i) remains within an acceptable range. Moreover, our scheme not only supports traitor tracing but also achieves unbounded vectors without requiring $i\mathcal{O}$, while avoiding the key escrow problem.

VI. CONCLUSION

To address the compatibility issue between “computation on ciphertexts” and “traceability” of data leaks of sensitive data on Web service platforms, we designed a traceable registered functional encryption for unbounded inner product scheme without $i\mathcal{O}$. As we know, our TRFE-UIP scheme is the first one that natively integrates unbounded inner product computation, registered key management and public black-box traceability.

In addition, it resists key escrow risk inherently while enabling precise identification of malicious key distributors without decrypting the ciphertext. In the standard model, we proved that the TRFE-UIP scheme satisfies the strong Sel*-SIM security and public black-box traceability under MDDH assumption. Compared with the existing mainstream IPFE schemes, our scheme not only obtains stronger security and more complete functional support, but also has better performance. Therefore, our TRFE-UIP scheme is well suited for deployment on large-scale Web service platforms. To enhance the resilience of the scheme against attacks in practical deployment, future research can reduce reliance on a single trusted entity by constructing a distributed CRS generation mechanism and a decentralized KC governance mechanism, while further achieving an adaptive SIM-secure RFE scheme.

REFERENCES

- [1] A. Sahai and B. Waters, “Fuzzy identity-based encryption,” in *Proc. EUROCRYPT*, 2005, pp. 457–473.
- [2] D. Francati, D. Friolo, M. Maitra, G. Malavolta, A. Rahimi, and D. Venturi, “Registered (inner-product) functional encryption,” in *Proc. ASIACRYPT*, 2023, pp. 98–133.
- [3] V. Goyal, A. Jain, and A. O’Neill, “Multi-input functional encryption with unbounded-message security,” in *Proc. ASIACRYPT*, 2016, pp. 531–556.

- [4] J. Tomida, "Tightly secure inner product functional encryption: Multi-input and function-hiding constructions," *Theor. Comput. Sci.*, vol. 833, pp. 56–86, 2020.
- [5] Z. Zhu, K. Zhang, J. Gong, and H. Qian, "Registered abe via predicate encodings," in *Proc. ASIACRYPT*, 2023, pp. 66–97.
- [6] X. Qian, H. Li, M. Hao, G. Xu, H. Wang, and Y. Fang, "Decentralized multi-client functional encryption for inner product with applications to federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5781–5796, Nov.–Dec. 2024.
- [7] X. Qian et al., "Privacy-preserving data evaluation via functional encryption, revisited," in *Proc. IEEE INFOCOM*, 2024, pp. 11–20.
- [8] D. Boneh, A. Sahai, and B. Waters, "Functional encryption: Definitions and challenges," in *Proc. IEEE Trans. Cloud Comput.*, 2011, pp. 253–273.
- [9] S. Goldwasser et al., "Multi-input functional encryption," in *Proc. EUROCRYPT*, 2014, pp. 578–602.
- [10] M. Abdalla, F. Bourse, A. De Caro, and D. Pointcheval, "Simple functional encryption schemes for inner products," in *Proc. PKC*, 2015, pp. 733–751.
- [11] M. Abdalla, F. Bourse, H. Marival, D. Pointcheval, A. Soleimanian, and H. Waldner, "Multi-client inner-product functional encryption in the random-oracle model," in *Proc. SCN*, 2020, pp. 525–545.
- [12] X. Liu, S. Liu, S. Han, and D. Gu, "Tightly CCA-secure inner product functional encryption scheme," *Theor. Comput. Sci.*, vol. 898, pp. 1–19, 2022.
- [13] J. Han, L. Chen, A. Hu, L. Chen, and J. Li, "Privacy-preserving decentralized functional encryption for inner product," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 1680–1694, Jul.–Aug. 2024.
- [14] S. Garg, C. Gentry, S. Halevi, M. Raykova, A. Sahai, and B. Waters, "Candidate indistinguishability obfuscation and functional encryption for all circuits," *SIAM J. Comput.*, vol. 45, no. 3, pp. 882–929, 2016.
- [15] S. Badrinarayanan, D. Gupta, A. Jain, and A. Sahai, "Multi-input functional encryption for unbounded arity functions," in *Proc. ASIACRYPT*, 2015, pp. 27–51.
- [16] J. Tomida and K. Takashima, "Unbounded inner product functional encryption from bilinear maps," in *Proc. ASIACRYPT*, 2018, pp. 609–639.
- [17] S. Agrawal, R. Goyal, and J. Tomida, "Multi-Input quadratic functional encryption from pairings," in *Proc. CRYPTO*, 2021, pp. 208–238.
- [18] J. Tomida, "Unbounded quadratic functional encryption and more from pairings," in *Proc. EUROCRYPT*, 2023, pp. 543–572.
- [19] U. Dowerah, S. Dutta, A. Mitrokovska, S. Mukherjee, and T. Pal, "Unbounded predicate inner product functional encryption from pairings," *J. Cryptol.*, vol. 36, no. 3, doi: [10.1007/s00145-023-09458-2](https://doi.org/10.1007/s00145-023-09458-2).
- [20] M. Yang, J. Gong, and H. Qian, "Improved unbounded inner-product functional encryption," *Theor. Comput. Sci.*, vol. 1014, 2024, Art. no. 114756, doi: [10.1016/j.tcs.2024.114756](https://doi.org/10.1016/j.tcs.2024.114756).
- [21] X. T. Do, D. H. Phan, and D. Pointcheval, "Traceable inner product functional encryption," in *Proc. RSA*, 2020, pp. 564–585.
- [22] F. Luo, S. Al-Kuwari, H. Wang, and W. Han, "Generic construction of trace-and-revoke inner product functional encryption," in *Proc. ESORICS*, 2022, pp. 259–282.
- [23] S. Dutta, T. Pal, A. K. Singh, and S. Mukhopadhyay, "Embedded identity traceable identity-based IPFE from pairings and lattices," *Cryptol. ePrint Arch.*, 2022.
- [24] M. Qiu, J. Han, F. Hao, C. Sun, and G. Wu, "Privacy-preserving and traceable functional encryption for inner product in cloud computing," *IEEE Trans. Cloud Comput.*, vol. 13, no. 02, pp. 667–679, Apr.–Jun. 2025.
- [25] Z. Zhu, J. Li, K. Zhang, J. Gong, and H. Qian, "Registered functional encryptions from pairings," in *Proc. EUROCRYPT*, 2024, pp. 373–402.
- [26] S. Hohenberger, G. Lu, B. Waters, and D. J. Wu, "Registered attribute-based encryption," in *Proc. EUROCRYPT*, 2023, pp. 511–542.
- [27] Y. Zhang, J. Chen, D. He, and Y. Zhang, "Bounded collusion-resistant registered functional encryption for circuits," in *Proc. ASIACRYPT*, 2024, pp. 32–64.
- [28] Q. Du, Q. Chu, J. Chen, M. H. Au, and D. He, "MultiReg-FE: Registered FE for unbounded inner-product and attribute-weighted sums," *Cryptol. ePrint Arch.*, 2024.
- [29] P. Branco, R. W. Lai, M. Maitra, G. Malavolta, A. Rahimi, and I. K. Woo, "Traitor tracing without trusted authority from registered functional encryption," in *Proc. ASIACRYPT*, 2024, pp. 33–66.
- [30] A. Escala, G. Herold, E. Kiltz, C. Ràfols, and J. Villar, "An algebraic framework for Diffie–Hellman assumptions," *J. cryptol.*, vol. 30, no. 1, pp. 242–288, 2017.
- [31] C. S. Jutla and A. Roy, "Shorter quasi-adaptive NIZK proofs for linear subspaces," *J. Cryptol.*, vol. 30, no. 4, pp. 1116–1156, 2017.
- [32] E. Kiltz and H. Wee, "Quasi-adaptive nizk for linear subspaces revisited," in *Proc. EUROCRYPT*, 2015, pp. 101–128.

- [33] P. Datta, T. Pal, and S. Yamada, "Registered FE beyond predicates: (attribute-based) linear functions and more," in *Proc. ASIACRYPT*, 2024, pp. 65–104.
- [34] MRACL Ltd., "MIRACL cryptographic SDK: Multiprecision integer and rational arithmetic cryptographic library," Version 7.0.0, Jun. 2012. [Online] Available: <https://github.com/miracrl/MIRACL>
- [35] P. S. Barreto and M. Naehrig, "Pairing-friendly elliptic curves of prime order," in *Proc. Int. workshop Sel. areas cryptogr.*, 2005, pp. 319–331.



Jinmei Tian received the B.S. degree in Computer Science from Hainan Normal University, China, in 2019 and the M.S. degree in Computer Science from Nanjing Normal University, China, in 2022. She is pursuing a Ph.D. degree in the College of Computer and Cyber Security, Fujian Normal University, China. Her research interests include information security and cryptography.



Jianting Ning (Senior Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China, in 2016. He is with the College of Computer and Cyber Security, Fujian Normal University, with the College of Information Science and Engineering, Zhejiang Sci-Tech University, and with the Faculty of Data Science, City University of Macau. His research interests include applied cryptography and information security.



Jian Shen (Member, IEEE) received the M.S. and Ph.D. degrees in computer science from Chosun University, South Korea, in 2009 and 2012, respectively. He now is the Dean of the School of Information Science and Engineering, Zhejiang Sci-Tech University, China. His research interests are public cryptography, cloud computing and security, data auditing and sharing, and information security systems.



Kai Zhang (Member, IEEE) received the B.S. degree in computer science and technology from Shandong Normal University, China, in 2012, and the Ph.D. degree in computer science and technology from East China Normal University, China, in 2017. Currently, he is a Professor with Shanghai University of Electric Power, China. His research interests include applied cryptography and information security.



Weijing You received the Ph.D. degree in information security from the University of Chinese Academy of Sciences, China, in 2021. She is currently an associate professor with the College of Computer and Cyber Security at the Fujian Normal University, China. Her research interests are in applied cryptography and data security. She is a senior member of China Computer Federation (CCF).



Lefeng Zhang received the B.S. and M.S. degrees from the Zhongnan University of Economics and Law, China, in 2016 and 2019, respectively, and the Ph.D. degree from the University of Technology Sydney, Australia, in 2024. He is currently working as an assistant professor in the Faculty of Data Science, City University of Macau, Macao SAR, China. His research interests include game theory and privacy preserving.